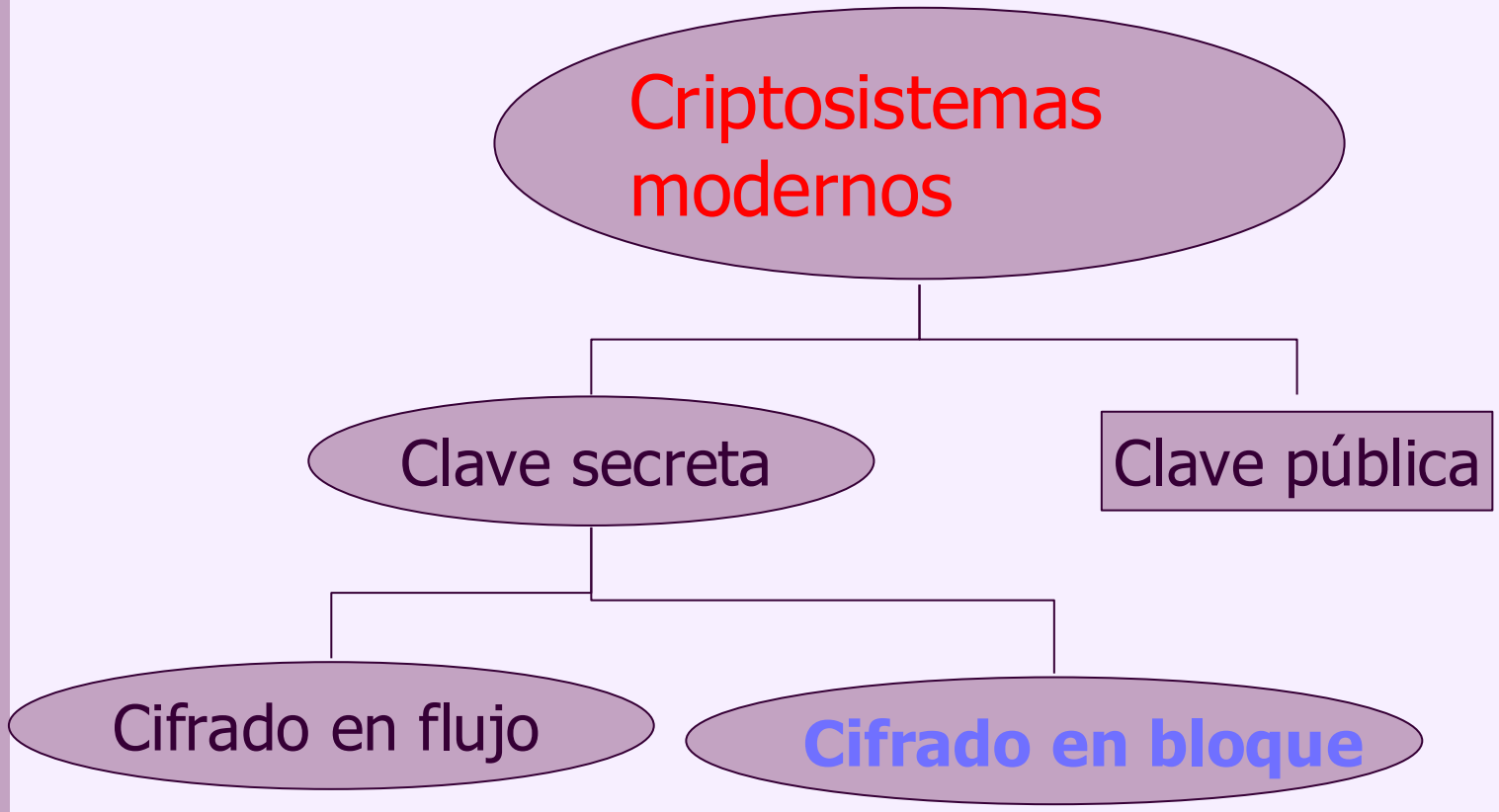
# Módulo II

## Comunicaciones seguras

- Tema 3: Protección de la confidencialidad
- Tema 4: Cifrados de clave secreta
- Tema 5: Distribución de claves

# Protección de la confidencialidad



# Comunicaciones Seguras

En:

- Wi-Fi
- SSL/TLS
- BLE
- WhatsApp y Telegram
- TOR
- Telefonía móvil
- DNI-e











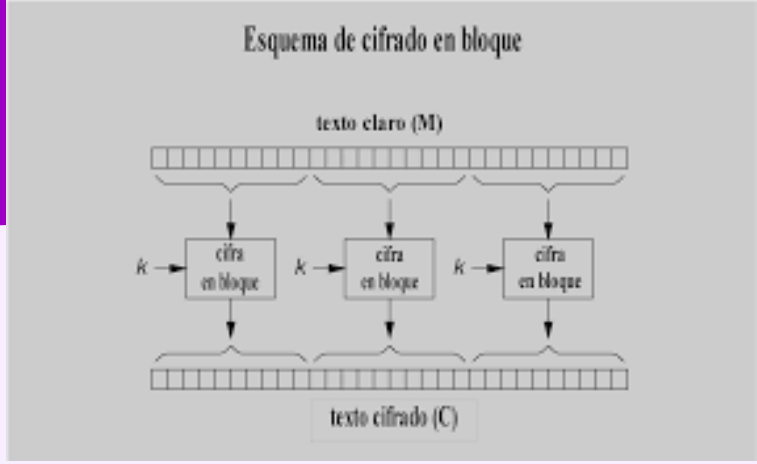








# Cifrado en Bloque




- El mensaje en claro se divide en bloques y se aplica el algoritmo sobre cada uno de forma que:
  - –el cifrado de cada símbolo depende de los demás símbolos del mismo bloque
  - –para descifrar una parte no hace falta descifrar el mensaje completo
- Dos elementos típicos de diseño:
  - Varias iteraciones
  - Expansión de claves

# Cifrados en Bloque

1. KASUMI

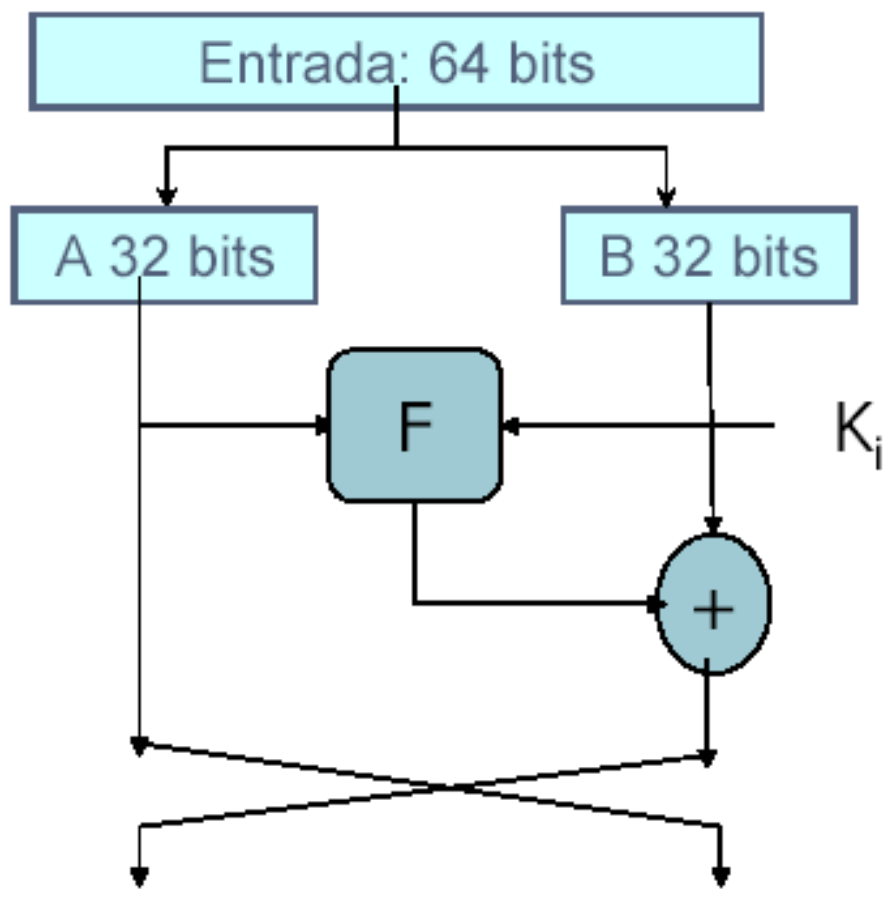



2. AES



# Cifrados Tipo Feistel

- Dado un bloque de N bits, se divide en dos mitades, y en cada iteración se realizan operaciones con la clave $k_i$ sólo sobre una mitad.
- En cada iteración se intercambian las dos mitades



# Cifrado KASUMI

- KASUMI, también llamado A5/3, es un cifrado en bloque utilizado en confidencialidad (f8) e integridad (f9) para **Telefonía móvil 3G (UMTS)**.
- KASUMI fue diseñado por Mitsubishi.
- **Usa bloques de 64 bits, y claves de 128 bits.**
- Es un cifrado **tipo Feistel** con **8 iteraciones,** estructura recursiva y subcomponentes también tipo Feistel

# Cifrado KASUMI

- En las iteraciones pares las operaciones se ejecutan en orden inverso a como se ejecutan en las impares.
- Se usan 40 subclaves de 16 bits, precalculadas y almacenadas.
- Muchas subclaves se calculan con un algoritmo de expansión de claves.
- Las restantes subclaves se generan mediante una XOR con constantes predefinidas.

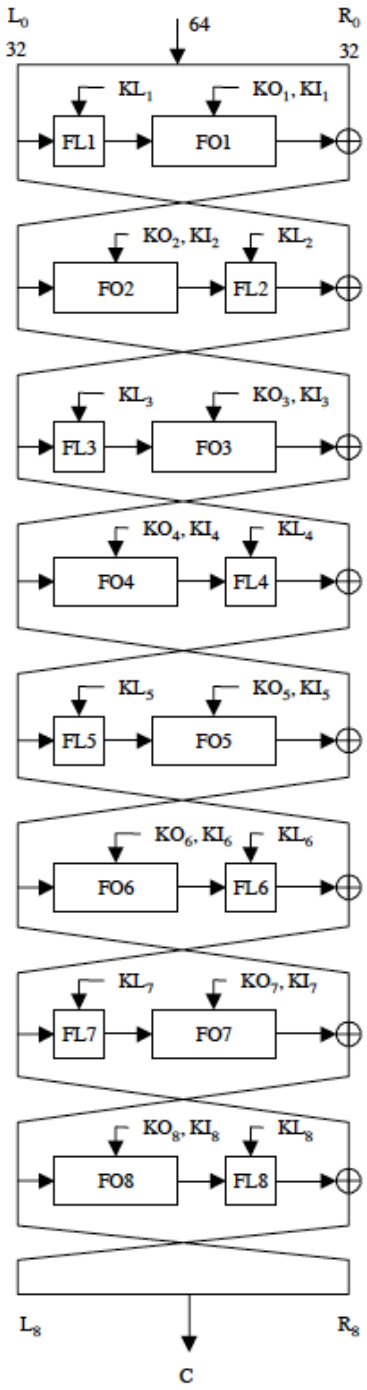



Fig. 1: KASUMI

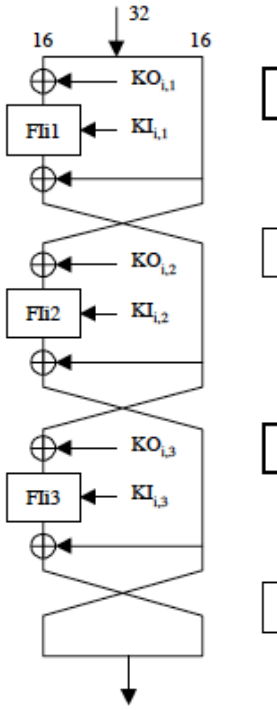



Fig.2: FO Function

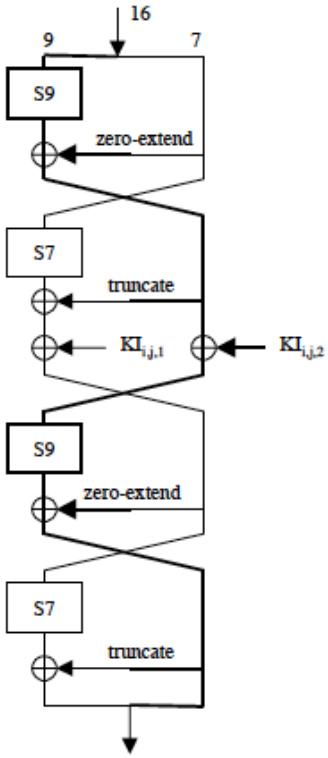



Fig.3: FI Function

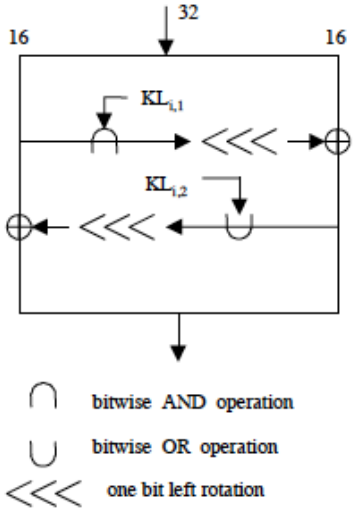



Fig.4: FL Function

P. Caballero Gil

# Expansión de claves en KASUMI

- La clave K de 128 bits se divide en 8 bloques de 16 bits:

$$K = K_1 \| K_2 \| K_3 \| K_4 \| K_5 \| K_6 \| K_7 \| K_8$$

- Otra clave K', modificación de K, se obtiene de la clave original por operación XOR con 0x123456789ABCDEFFEDCBA9876543210, y se divide también en 8 bloques de 16 bits.
- Las subclaves se obtienen o bien de la clave modificada o bien de otras subclaves mediante rotación a izquierda:

$$\begin{aligned}
KL_{i,1} &= \mathrm{ROL}(K_i, 1) \\
KL_{i,2} &= K'_{i+2} \\
KO_{i,1} &= \mathrm{ROL}(K_{i+1}, 5) \\
KO_{i,2} &= \mathrm{ROL}(K_{i+5}, 8) \\
KO_{i,3} &= \mathrm{ROL}(K_{i+6}, 13) \\
KI_{i,1} &= K'_{i+4} \\
KI_{i,2} &= K'_{i+3} \\
KI_{i,3} &= K'_{i+7}
\end{aligned}$$

# Cifrado KASUMI

- Los bloques de 64 se procesan bits en dos mitades de 32 bits.
- En cada iteración, a la mitad derecha se le hace XOR con la salida de una función

$$\begin{aligned} L_i &= F_i(KL_i, KO_i, KI_i, L_{i-1}) \oplus R_{i-1} \\ R_i &= L_{i-1} \end{aligned}$$

- Las funciones son diferentes para iteraciones pares e impares
- En cada caso, es una composición de dos funciones FL y FO.
- En impares es: $F_i(K_i, L_{i-1}) = FO(KO_i, KI_i, FL(KL_i, L_{i-1}))$
- En pares es: $F_i(K_i, L_{i-1}) = FL(KL_i, FO(KO_i, KI_i, L_{i-1}))$
- Ambas funciones FL y FO dividen los datos de entrada de 32 bits en dos mitades de 16 bits.
- La función FL está formada solo por operaciones lógicas, desplazamientos binarios y cajas S.
- La función FO es una función Feistel de 3 iteraciones.

# Seguridad de KASUMI

- En 2001 se presentó un ataque diferencial contra KASUMI con 6 iteraciones.
- En 2005, se publicó un ataque boomerang contra 8 iteraciones de KASUMI, y con texto original escogido.
- En 2010 con un ordenador se logró descifrar una clave completa de 128 bits en dos horas, pero para hacerlo necesitó tener control completo de las entradas y salidas del algoritmo.
- **Todavía se considera seguro para su uso en telefonía móvil 3G.**

# Cifrado Rijndael (AES)

- 1. RIJNDAEL, 86 votos.
- 2. SERPERNT, 59 votos.
- 3. TWOFISH, 31 votos.
- 4. RC6, 23 votos.
- 5. MARS, 13 votos.

- En 1996 el NIST (National Institute of Standards and Technology) inició el proceso de selección del AES (Advanced Encryption Standard) para sustituir al DES.
- En 2000 ganó RijnDael, de los belgas **Vincent Rijmen** y Daemen
- Declarado **estándar hasta 2060**.
- Se usa en:
  - navegación segura con TLS
  - Wifi WPA2
  - BLE
  - WhatsApp y Telegram
  - TOR
  - Telefonía móvil 4G y 5G
  - DNI-e



# Requisitos y Criterios en la Selección del AES

**Requisitos:**

- Longitud de bloque = 128 bits
- Longitud de clave = 128, 192 y 256 bits
- Número de iteraciones flexible: 10, 12, 14
- Operaciones a nivel de byte



**Criterios:**

- Esfuerzo criptoanalítico
- Eficacia computacional
- Hardware y software
- Simplicidad de diseño
- Flexibilidad
- No de tipo Feistel
- Fácilmente paralelizable
- Implementación en smartcard
- Sin requisitos de licencia

# Cifrado AES

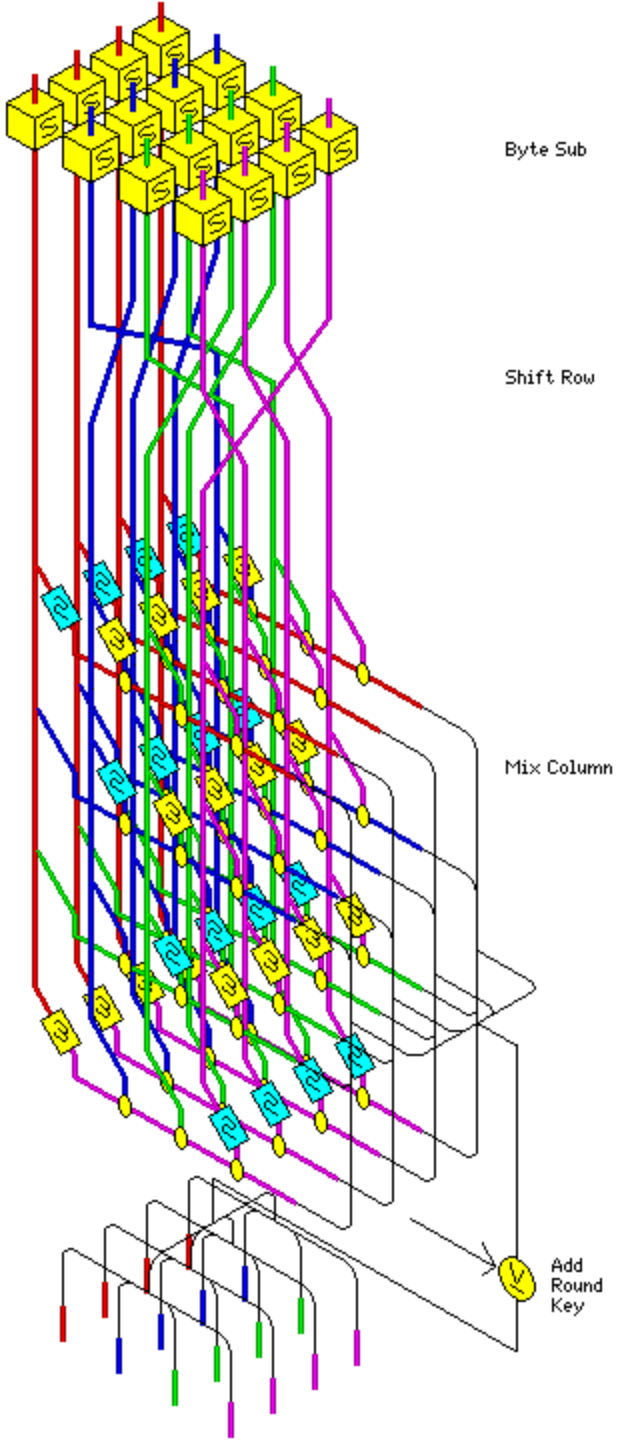
- Cálculos en GF($2^8$), e.d. con bytes.
- En cada iteración se realizan 4 pasos:

confusión 1. **Sustitución** de cada byte por su recíproco

difusión 2. **Desplazamiento** de bytes

difusión 3. **Multiplicación** de cada columna por una matriz

confusión 4. **Suma** de la subclave y la información del estado intermedio actual



# Multiplicación en AES

- La multiplicación del AES se corresponde con un producto de dos polinomios en módulo $x^8+x^4+x^3+x+1$, que puede implementarse como desplazamientos de bytes, y XORs con el byte $1B_{16}= 00011011_2$.
- Implica aplicar operación distributiva sobre los dos bytes multiplicandos, usando para ello el byte de menor peso. Luego, para cada bit 1 de ese byte, desplazar a izquierda el otro byte, de forma que cada vez que su bit más significativo antes del desplazamiento sea 1, hay que hacer, tras el desplazamiento, una XOR con el byte $1B_{16}$.
- El nº de desplazamientos a realizar viene dado por la posición de los bits 1 en el 1er byte multiplicando.

# Multiplicación en AES

$$b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 b_3x^3 + b_2x^2 + b_1x + b_0.$$

$$\Updownarrow$$

$$(b_7b_6b_5b_4b_3b_2b_1b_0).$$

Los bytes se representan en forma de polinomios.

**Ej: $57_{(16}$= 0101 0111$_{(2}$= $x^6+x^4+x^2+x+1$ $83_{(16}$= 1000 0011$_{(2}$= $x^7+x+1$**

**Multiplicación en GF($2^8$ )**

Consiste en multiplicar los polinomios (en módulo 2), y el resultado reducirlo módulo el polinomio **$x^8+x^4+x^3+x+1$** (que se corresponde con el byte **00011011=1B**).

**Ej: {57} . {83} = {C1}**

$$(5\ 7)_{16} \otimes (8\ 3)_{16} = (C\ 1)_{16} \rightarrow$$
$$(x^6 + x^4 + x^2 + x\ \ +1) \otimes (x^7 + x\ \ +1) =$$
$$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 \equiv$$
$$(x^7 + x^6 + 1) \bmod (x^8 + x^4 + x^3 + x\ \ +1)$$

**Ej: 01010111x10000011=** 01010111x00000001+01010111x00000010+01010111x10000000=

01010111+10101110+01010111x10000000=01010111+10101110+00111000= **11000001=C1**

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| **01010111** | 10101110 | 01011100+00011011= 01000111 | 10001110 | 00011100+00011011= 00000111 | 00001110 | 00011100 | **00111000** |



# Cifrado AES

- Cada operación del algoritmo tiene como entrada la salida de la operación anterior. Este resultado intermedio se llama estado.
- Lo vemos para el caso de longitud de bloque 128.
- El estado se representa como una matriz de 4 filas y Nb=4 columnas de bytes

| | | | |
|---|---|---|---|
| $a_{0,0}$ | $a_{0,1}$ | $a_{0,2}$ | $a_{0,3}$ |
| $a_{1,0}$ | $a_{1,1}$ | $a_{1,2}$ | $a_{1,3}$ |
| $a_{2,0}$ | $a_{2,1}$ | $a_{2,2}$ | $a_{2,3}$ |
| $a_{3,0}$ | $a_{3,1}$ | $a_{3,2}$ | $a_{3,3}$ |

- La clave también se representa con una matriz de 4 filas y Nk=4 columnas de bytes

| | | | |
|---|---|---|---|
| $k_{0,0}$ | $k_{0,1}$ | $k_{0,2}$ | $k_{0,3}$ |
| $k_{1,0}$ | $k_{1,1}$ | $k_{1,2}$ | $k_{1,3}$ |
| $k_{2,0}$ | $k_{2,1}$ | $k_{2,2}$ | $k_{2,3}$ |
| $k_{3,0}$ | $k_{3,1}$ | $k_{3,2}$ | $k_{3,3}$ |

# Cifrado AES

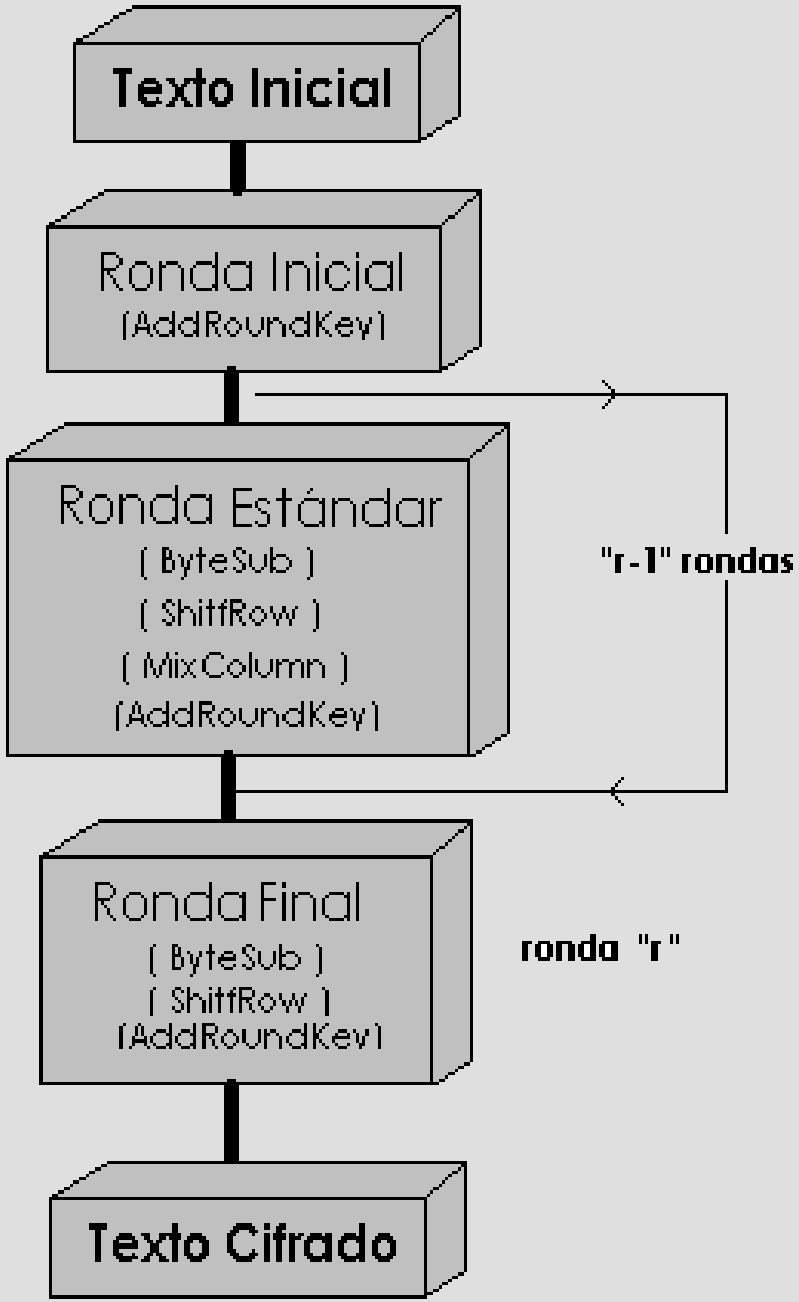
El **número de iteraciones** que se utilizan depende de la versión del algoritmo que utilicemos, es decir:

| | $N_b = 4$ (128 bits) | $N_b = 6$ (192 bits) | $N_b = 8$ (256 bits) |
|---|---|---|---|
| $N_k = 4$ (128 bits) | 10 | 12 | 14 |
| $N_k = 6$ (192 bits) | 12 | 12 | 14 |
| $N_k = 8$ (256 bits) | 14 | 14 | 14 |

Número de rondas para AES en función de los tamaños de clave y bloque.

En el caso que estudiamos, (longitudes de bloque y clave 128), los nº de columnas son Nb=Nk=4 y el nº de iteraciones es 10.

-Cuando la clave es de 256 bits el procedimiento es diferente a cuando es de 128 y 192 bits.




P. Caballero Gil

# Iteración 0: Transformación AddRoundKey en AES

Iteración 0

**ARK: AddRoundKey:**

Consiste en una OR-EXCLUSIVA entre el bloque de entrada y la clave original de cifrado. El resultado es el ESTADO INTERMEDIO 1

Ejemplo:

Dado el bloque de entrada y la clave de 128 bits siguientes:

**Bloque de entrada = 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34**

**Clave = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c**

Se suman los dos elementos:

| | | | |
|---|---|---|---|
| 32 | 88 | 31 | e0 |
| 43 | 5a | 31 | 37 |
| f6 | 30 | 98 | 07 |
| a8 | 8d | a2 | 34 |

Bloque de entrada

⊕

| | | | |
|---|---|---|---|
| 2b | 28 | ab | 09 |
| 7e | ae | f7 | cf |
| 15 | D2 | 15 | 4f |
| 16 | A6 | 88 | 3c |

Clave

=

| | | | |
|---|---|---|---|
| 19 | a0 | 9a | e9 |
| 3d | f4 | c6 | f8 |
| e3 | e2 | 8d | 48 |
| be | 2b | 2a | 08 |

Estado intermedio 1



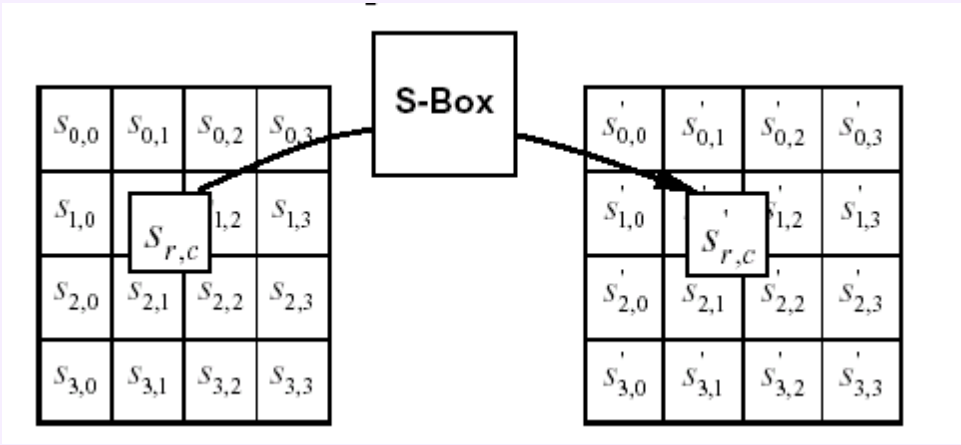
https://www.cryptool.org/en/cto/aes-step-by-step

# Transformación SubBytes en AES

## Transformación 1. ByteSub

Sustitución **no lineal** de los bytes de la matriz de estado basada en una S-Caja que para cada byte genera un nuevo byte.

Ejemplo:




| | | | |
|---|---|---|---|
| 19 | a0 | 9a | e9 |
| 3d | f4 | c6 | f8 |
| e3 | e2 | 8d | 48 |
| be | 2b | 2a | 08 |

Estado intermedio 1

⇒

| | | | |
|---|---|---|---|
| d4 | e0 | b8 | 1e |
| 27 | bf | b4 | 41 |
| 11 | 98 | 5d | 52 |
| ae | f1 | e5 | 30 |

Estado intermedio 2

# Transformación SubBytes en AES

| x \ y | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
| 1 | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
| 2 | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
| 3 | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
| 4 | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
| 5 | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
| 6 | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
| 7 | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
| 8 | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
| 9 | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
| a | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
| b | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | ea | 65 | 7a | ae | 08 |
| c | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
| d | 70 | 3e | b5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
| e | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
| f | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | b0 | 54 | bb | 16 |

# Transformación SubBytes en AES

- Se trata de una **caja de sustitución S** que actúa sobre bytes

$$(x\ y)_{16} \Rightarrow (x'\ y')_{16} \qquad (4\ 5)_{16} \Rightarrow (6\ E)_{16}$$

- Se hizo público el criterio de diseño de la caja S
  - Se escribe cada byte en forma de polinomio

$$(5\ 7)_{16} \rightarrow (0101\ \ 0111)_2 \rightarrow x^6 + x^4 + x^2 + x + 1$$

  - Se calcula su inverso multiplicativo en GF($2^8$)

$$c_7x^7 + c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + 1$$

# Transformación SubBytes en AES

– Se aplica la transformación lineal

$$\begin{bmatrix} y'_0 \\ y'_1 \\ y'_2 \\ y'_3 \\ x'_0 \\ x'_1 \\ x'_2 \\ x'_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



$$(x'_3, x'_2, x'_1, x'_0, y'_3, y'_2, y'_1, y'_0) = (x'\ y')_{16}$$

$$(5\ 7)_{16} \Rightarrow (5\ B)_{16}$$

# Transformación ShiftRow en AES

## Transformación 2. ShiftRow

- Consiste en desplazar a la izquierda los bytes de las filas que conforman la matriz del estado actual.
- Cada fila i=0,1,2,3 se desplaza un nº i de posiciones diferentes

**Continuando el ejemplo**

1. La fila 0 no es rotada
2. La fila 1 es rotada 1 byte
3. La fila 2 es rotada 2 bytes
4. La fila 3 es rotada 3 bytes

| | | | |
|---|---|---|---|
| d4 | e0 | b8 | 1e |
| 27 | bf | b4 | 41 |
| 11 | 98 | 5d | 52 |
| ae | f1 | e5 | 30 |

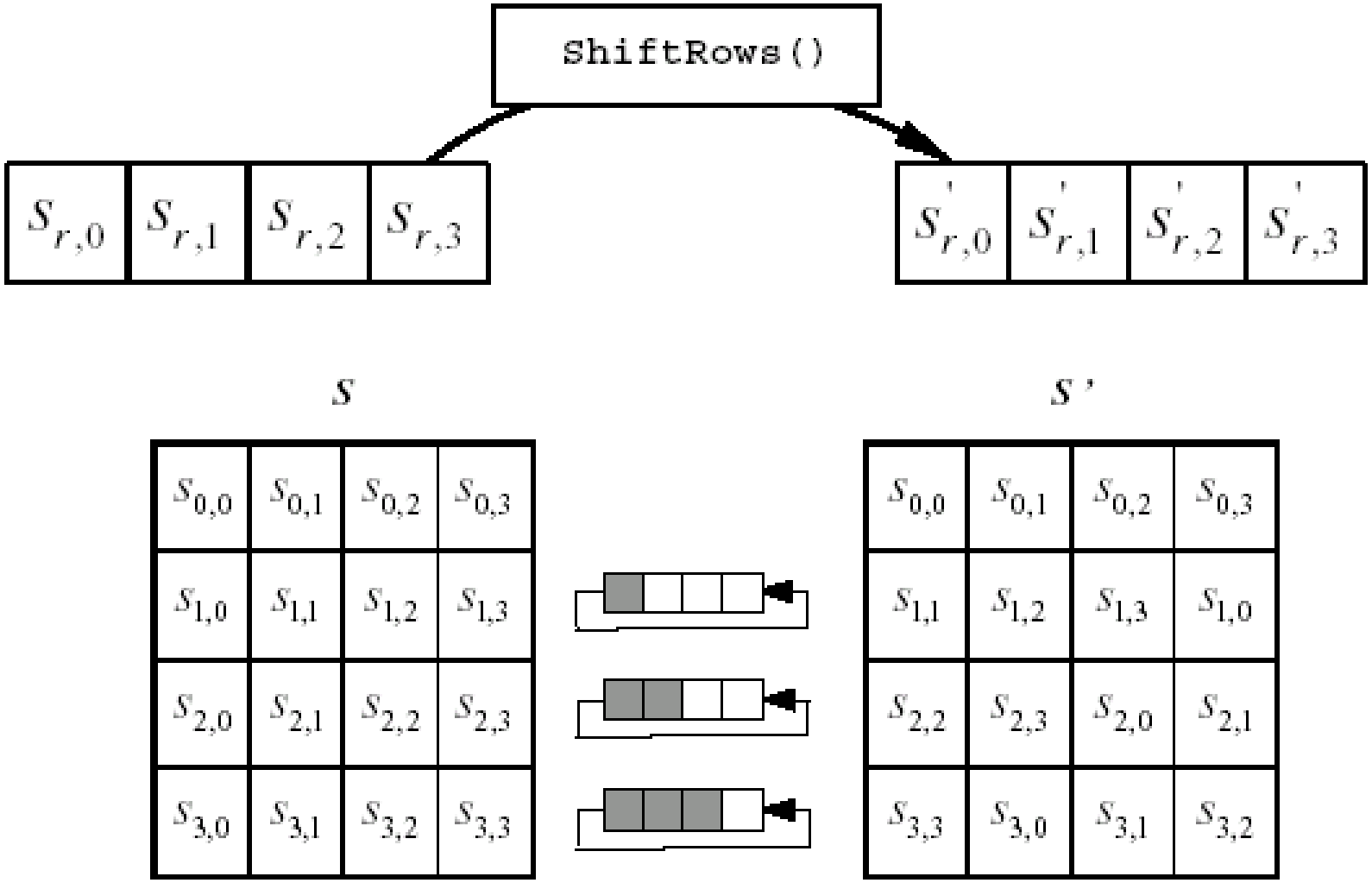
Estado intermedio 2

⇒

| | | | |
|---|---|---|---|
| d4 | e0 | b8 | 1e |
| bf | b4 | 41 | 27 |
| 5d | 52 | 11 | 98 |
| 30 | ae | f1 | e5 |

Estado intermedio 3

# Transformación ShiftRow en AES



# Transformación MixColumn en AES

## Transformación 3. MixColumn

Visto de forma matricial, donde "c" representa el índice de la columna que se procesa:

Ejemplo:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$$

$$s'_{1,c} = s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c}$$

$$s'_{2,c} = s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c})$$

$$s'_{3,c} = (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c})$$

| | | | |
|---|---|---|---|
| d4 | e0 | b8 | 1e |
| bf | b4 | 41 | 27 |
| 5d | 52 | 11 | 98 |
| 30 | ae | f1 | e5 |

Estado intermedio 3

⇒

| | | | |
|---|---|---|---|
| 04 | e0 | 48 | 28 |
| 66 | cb | f8 | 06 |
| 81 | 19 | d3 | 26 |
| e5 | 9a | 7a | 4c |

Estado intermedio 4

# Transformación MixColumn en AES

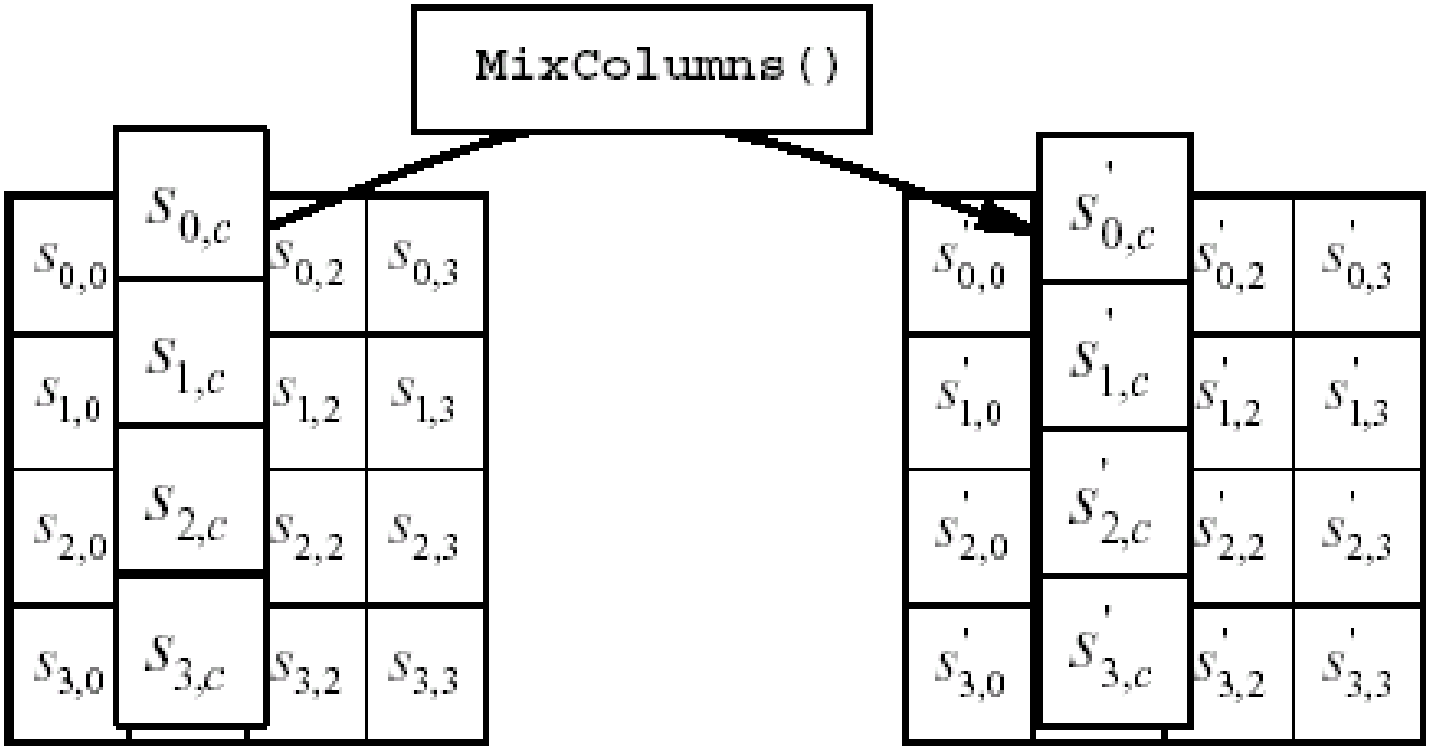
$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{for } 0 \le c < Nb.$$

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$$

$$s'_{1,c} = s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c}$$

$$s'_{2,c} = s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c})$$

$$s'_{3,c} = (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c}).$$

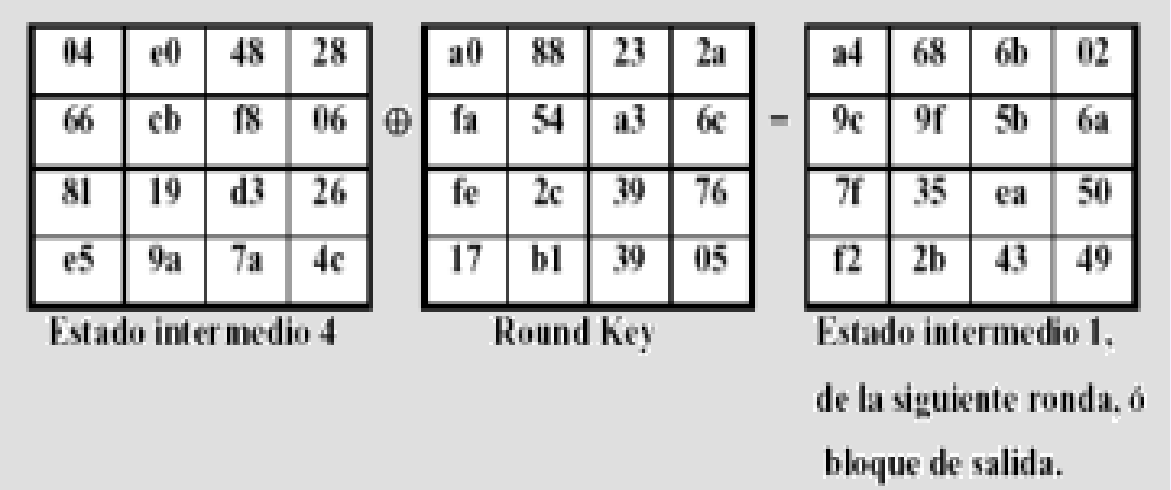


# Transformación AddRoundKey en AES

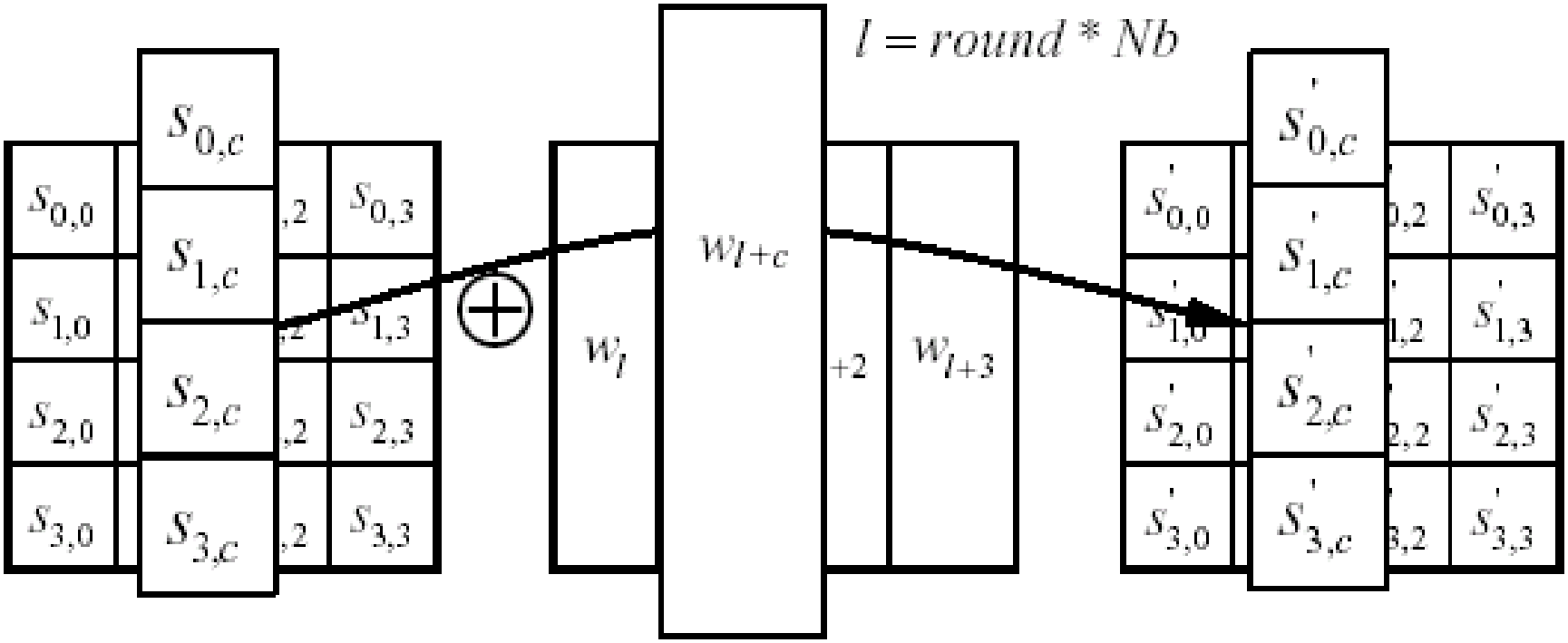
## Transformación 4. AddRoundKey

Esta transformación consiste en una OR-EXCLUSIVO entre el Estado Intermedio 4 y una subclave que se genera a partir de la clave original de cifrado.

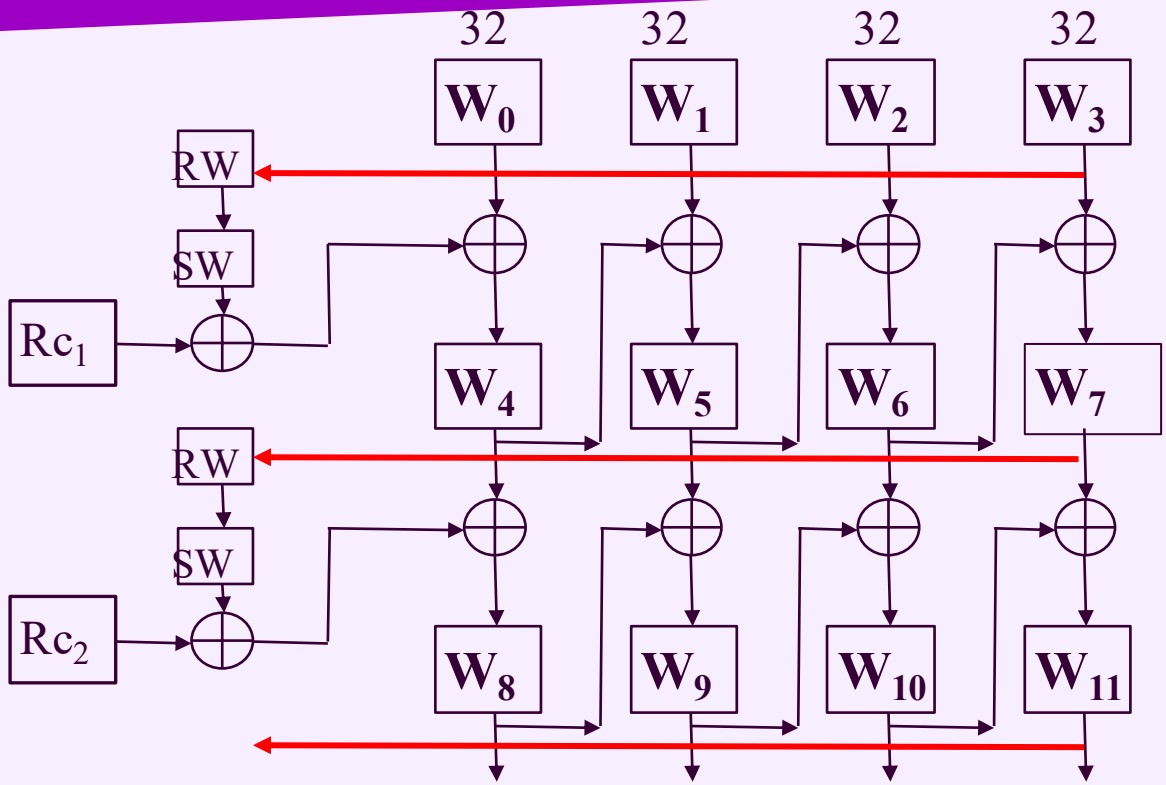
Ejemplo:



# Expansión de Claves en AES



# Expansión de Claves en AES




The first four words (w[0] through w[3]) are the input key.
To calculate w[i] for i from 4 to 39:
1.Set temp = w[i-1]
2.If i = 4, 8, 12, 16, ..., 36 (multiples of 4), temp is transformed by:
    1. Rotate this word left one byte cyclically
    2. Substitute each byte (using the same S-box of SubBytes)
    3. Xor with the round constant Rcon[i]
3.Set w[i] = w[i-4] xor temp

# Expansión de claves en AES

- **RW** rotación cíclica a izquierdas de los 4 bytes de entrada

$$RW(b_0, b_1, b_2, b_3) = (b_1, b_2, b_3, b_0)$$

- **SW** se aplica la caja S a cada uno de los 4 bytes de entrada

$$SW(b_1, b_2, b_3, b_0) = (b_1', b_2', b_3', b_0')$$

- Luego se suma bit a bit una constante $Rc_i$ y el resultado anterior

$$Rc_i \oplus (RW(SW(W)))$$

# Expansión de claves en AES

$Rc_1$= 01 00 00 00
$Rc_2$= 02 00 00 00
$Rc_3$= 04 00 00 00
$Rc_4$= 08 00 00 00
$Rc_5$= 10 00 00 00
$Rc_6$= 20 00 00 00
$Rc_7$= 40 00 00 00
$Rc_8$= 80 00 00 00
$Rc_9$= 1B 00 00 00
$Rc_{10}$= 36 00 00 00

https://www.youtube.com/watch?v=mlzxpkdXP58

http://www.formaestudio.com/rijndaelinspector/archivos/inspector.html

# Expansión de claves en AES

- Con la clave K, gracias a la expansión de clave, se genera un total de Nb*(Nr + 1) palabras de 4 bytes pues en cada una de las Nr+1 iteraciones hace falta una subclave de Nb*4 bytes.
- El resultado consiste en una serie lineal de palabras de 4 bytes, denotado Wi, 0<= i <Nb (Nr + 1).
- Las primeras Nk palabras de la clave expandida se llenan con la clave.
- Cada palabra siguiente Wi es igual a la XOR de la palabra anterior W(i-1) y la palabra W(i-Nk).
- Para las palabras en posiciones que son un múltiplo de Nk antes de esa XOR se aplica una transformación a W(i-1).
  - **Cuando la clave es de 256 bits** (Nk = 8) el procedimiento es ligeramente diferente a cuando es de 128 y 192 bits pues si Nk = 8 e i-4 es múltiplo de Nk, hay que aplicar la caja S a W(i-1) antes de la XOR.

# Expansión de claves en AES

```
KeyExpansion(byte key[4*Nk], word w[Nb*(Nr+1)], Nk)
begin
    word  temp

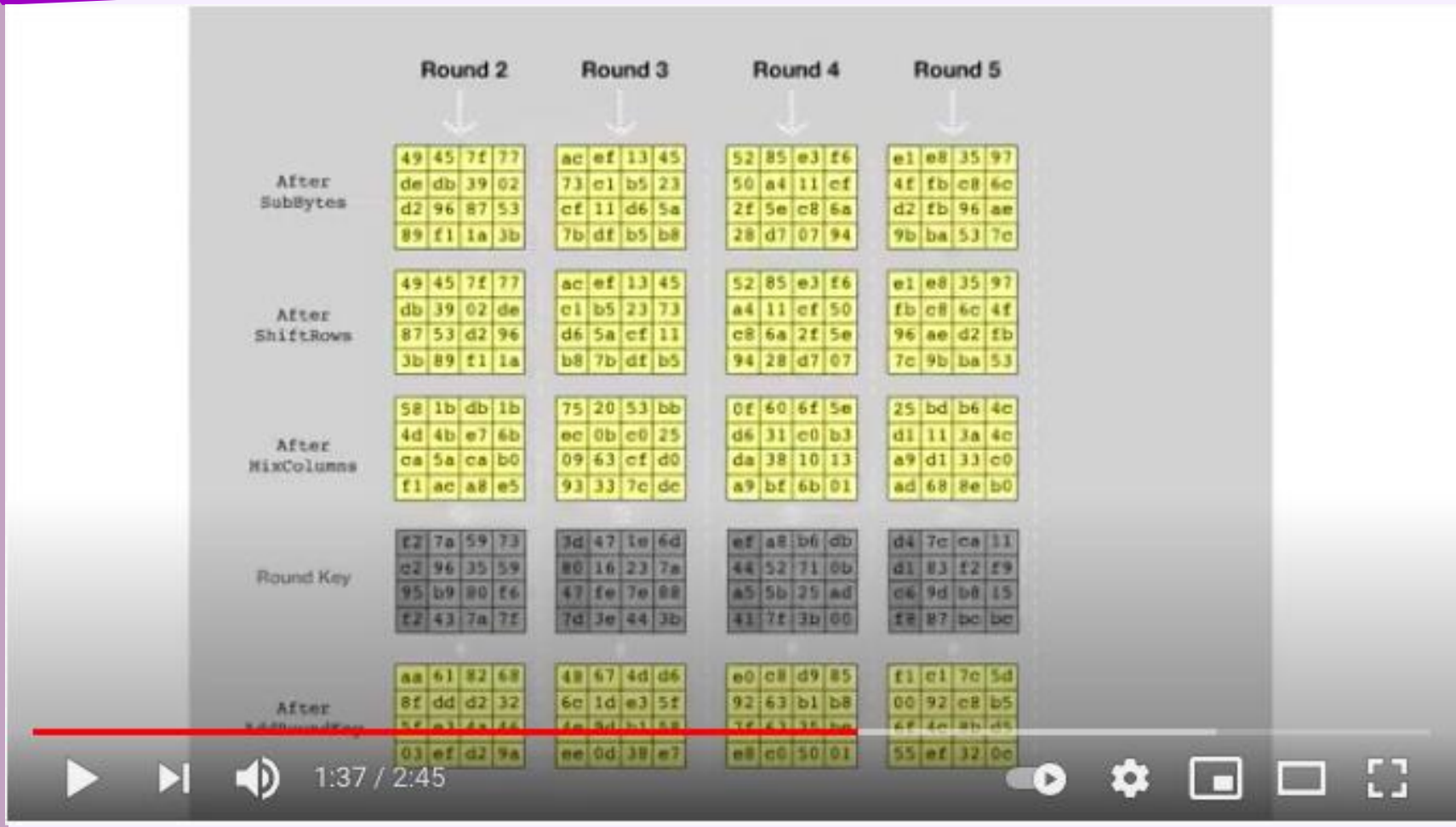
    i = 0

    while (i < Nk)
        w[i] = word(key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
        i = i+1
    end while

    i = Nk

    while (i < Nb * (Nr+1)]
        temp = w[i-1]
        if (i mod Nk = 0)
            temp = SubWord(RotWord(temp)) xor Rcon[i/Nk]
        else if (Nk > 6 and i mod Nk = 4)
            temp = SubWord(temp)
        end if
        w[i] = w[i-Nk] xor temp
        i = i + 1
    end while
end
```

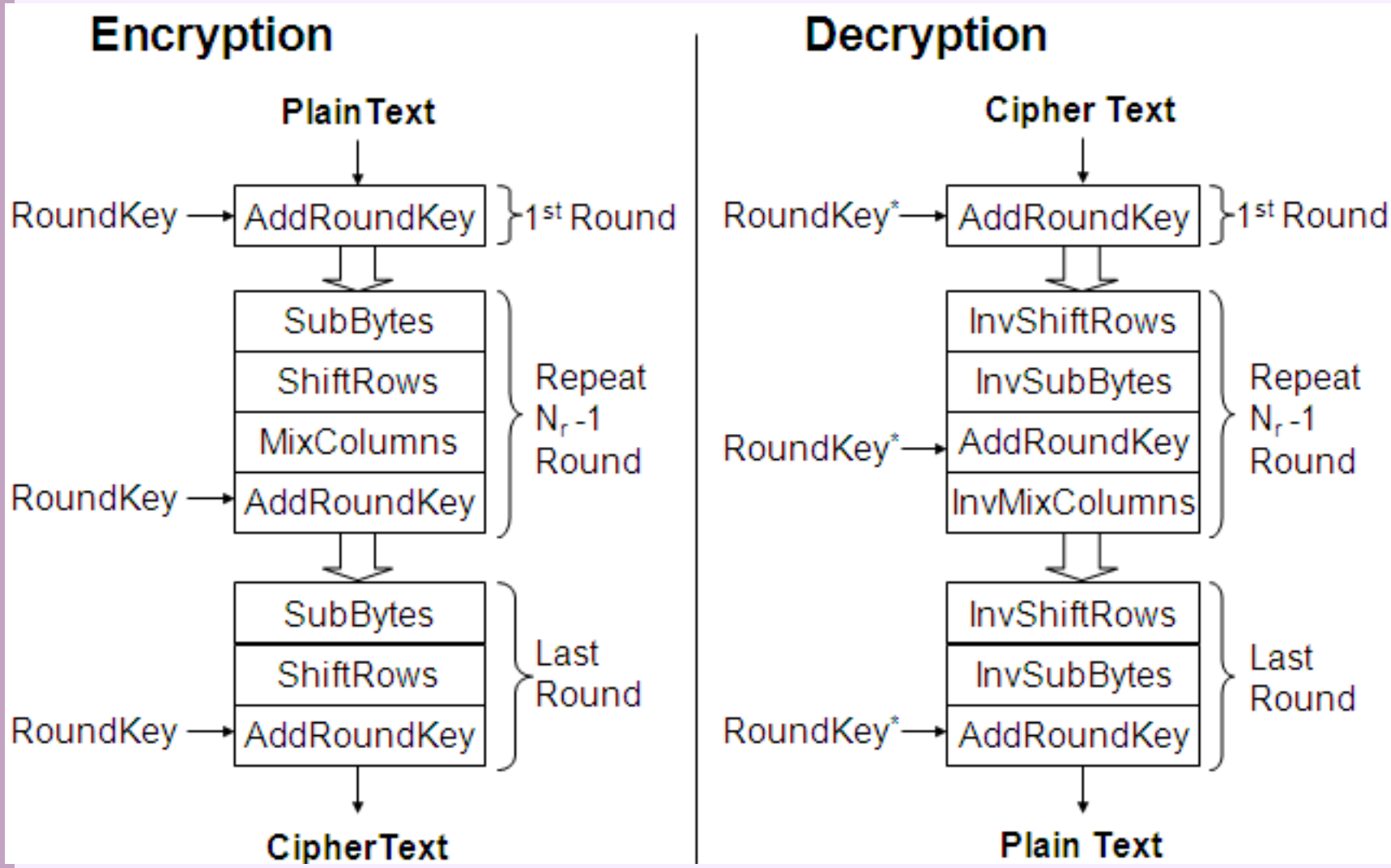
# Simulación del AES




https://www.youtube.com/watch?v=mlzxpkdXP58

http://www.formaestudio.com/rijndaelinspector/archivos/inspector.html

# Descifrado en AES



# Descifrado en AES

```
InvCipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
   byte  state[4,Nb]

   state = in

   AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1]) // See Sec. 5.1.4

   for round = Nr-1 step -1 downto 1
      InvShiftRows(state)                    // See Sec. 5.3.1
      InvSubBytes(state)                     // See Sec. 5.3.2
      AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
      InvMixColumns(state)                   // See Sec. 5.3.3
   end for

   InvShiftRows(state)
   InvSubBytes(state)
   AddRoundKey(state, w[0, Nb-1])

   out = state
end
```

# Descifrado en AES

## Descifrado:

Es similar al de cifrado, pero algunas operaciones son las transformaciones inversas de las operaciones realizadas en el cifrado, y las subclaves se usan en orden inverso.

Transformación 1. InverseByteSub (S-Caja):

| Hex | | y | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | F |
| x | 0 | 52 | 09 | 6a | d5 | 30 | 36 | a5 | 38 | bf | 40 | a3 | 9e | 81 | f3 | d7 | fb |
| | 1 | 7c | e3 | 39 | 82 | 9b | 2f | ff | 87 | 34 | 8e | 43 | 44 | c4 | de | e9 | cb |
| | 2 | 54 | 7b | 94 | 32 | a6 | c2 | 23 | 3d | ee | 4c | 95 | 0b | 42 | fa | c3 | 4e |
| | 3 | 08 | 2e | a1 | 66 | 28 | d9 | 24 | b2 | 76 | 5b | a2 | 49 | 6d | 8b | d1 | 25 |
| | 4 | 72 | f8 | f6 | 64 | 86 | 68 | 98 | 16 | d4 | a4 | 5c | cc | 5d | 65 | b6 | 92 |
| | 5 | 6c | 70 | 48 | 50 | fd | ed | b9 | da | 5e | 15 | 46 | 57 | a7 | 8d | 9d | 84 |
| | 6 | 90 | d8 | ab | 00 | 8c | bc | d3 | 0a | f7 | e4 | 58 | 05 | b8 | b3 | 45 | 06 |
| | 7 | d0 | 2c | 1e | 8f | ca | 3f | 0f | 02 | c1 | af | bd | 03 | 01 | 13 | 8a | 6b |
| | 8 | 3a | 91 | 11 | 41 | 4f | 67 | dc | ea | 97 | f2 | cf | ce | f0 | b4 | e6 | 73 |
| | 9 | 96 | ac | 74 | 22 | e7 | ad | 35 | 85 | e2 | f9 | 37 | e8 | 1c | 75 | df | 6e |
| | A | 47 | f1 | 1a | 71 | 1d | 29 | c5 | 89 | 6f | b7 | 62 | 0e | aa | 18 | be | 1b |
| | B | fc | 56 | 3e | 4b | c6 | d2 | 79 | 20 | 9a | db | c0 | fe | 78 | cd | 5a | f4 |
| | C | 1f | dd | a8 | 33 | 88 | 07 | c7 | 31 | b1 | 12 | 10 | 59 | 27 | 80 | ec | 5f |
| | D | 60 | 51 | 7f | a9 | 19 | b5 | 4a | 0d | 2d | e5 | 7a | 9f | 93 | c9 | 9c | ef |
| | E | a0 | e0 | 3b | 4d | ae | 2a | f5 | b0 | c8 | eb | bb | 3c | 83 | 53 | 99 | 61 |
| | f | 17 | 2b | 04 | 7e | ba | 77 | d6 | 26 | e1 | 69 | 14 | 63 | 55 | 21 | 0c | 7d |

– Tabla de Substitución inversa – InvSbox [xy] (en hexadecimal)

# Descifrado en AES

## Transformación 2. InverseShiftRow

Se realiza el mismo proceso que en el cifrado pero desplazando a la derecha los bytes de las filas que conforman la matriz del estado actual.

## Transformación 3. InverseMixColumn

$$s_{0,c} = (\{0e\} \bullet s'_{0,c}) \oplus (\{0b\} \bullet s'_{1,c}) \oplus (\{0d\} \bullet s'_{2,c}) \oplus (\{09\} \bullet s'_{3,c})$$

$$s_{1,c} = (\{09\} \bullet s'_{0,c}) \oplus (\{0e\} \bullet s'_{1,c}) \oplus (\{0b\} \bullet s'_{2,c}) \oplus (\{0d\} \bullet s'_{3,c})$$

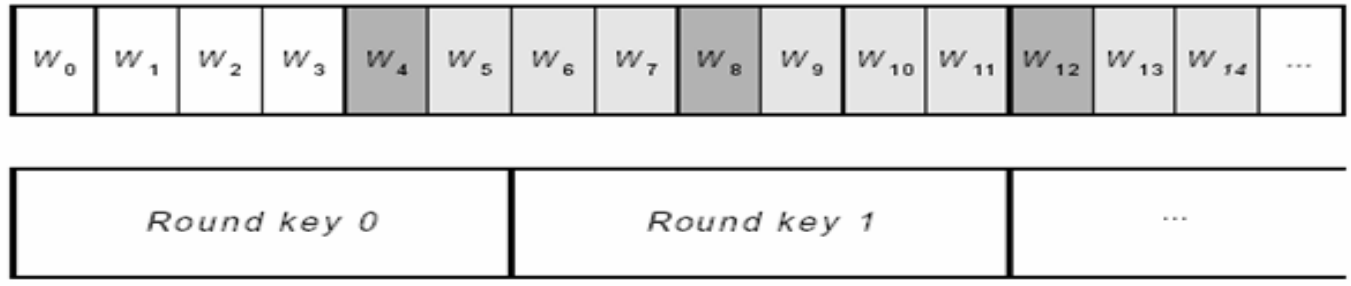
$$s_{2,c} = (\{0d\} \bullet s'_{0,c}) \oplus (\{09\} \bullet s'_{1,c}) \oplus (\{0e\} \bullet s'_{2,c}) \oplus (\{0b\} \bullet s'_{3,c})$$

$$s_{3,c} = (\{0b\} \bullet s'_{0,c}) \oplus (\{0d\} \bullet s'_{1,c}) \oplus (\{09\} \bullet s'_{2,c}) \oplus (\{0e\} \bullet s'_{3,c})$$

# Descifrado en AES

## Generación de Subclaves

El proceso de selección de la clave para la iteración i, será tomada de la subclave desde w[Nb*i] hasta w [Nb *(i+1) ]



# Seguridad de AES

- El diseño de los parámetros del AES como: número de iteraciones, número de rotaciones de bytes, valores de las S-Cajas, etc, fueron elegidos para hacer al AES seguro frente a ataques conocidos como:
  - Criptoanálisis Diferencial
  - Criptoanálisis Lineal

- Contra algunas implementaciones de AES se han lanzado **ataques de canal lateral**.

- No existe ningún mecanismo práctico para romper el algoritmo AES, que sea más eficiente que un ataque por fuerza bruta; y, dada la longitud de clave, la fuerza bruta ahora mismo no es práctica.

# Cifrados en Wi-Fi

- **WEP** (RC4) desde 1997
- **WPA** (RC4) desde 2003
- **WPA2** (AES y RC4) desde 2006
- **WPA3** (AES) desde 2018 **en WiFi 6**



# Ataque NoMore a WPA

En 2015 se publicó un ataque práctico contra el algoritmo RC4 llamado Ataque NoMore, que:

- contra WPA-TKIP permite descifrar e inyectar paquetes arbitrarios en una hora, y
- contra TLS permite descifrar una cookie HTTPs en 75 horas.




https://www.elladodelmal.com/2015/07/rc4-no-more-robo-de-cookies-en-httpstls.html

# Ataque KRACK a WPA2

- **En 2017 se publicó el ataque KRACK** (Key Reinstallation AttaCK) para romper WPA2 basándose en una debilidad del estándar.
- Es un ataque de **reproducción** o reinyección en el que una transmisión interceptada es maliciosamente repetida para forzar la **reinstalación de la clave** compartida, con objeto de descifrar luego los datos que transmite la víctima.
- Ataca el **handshake** que se realiza cuando un usuario se conecta a la red, cuando se verifican las credenciales de cliente y punto de acceso. Una vez acabado el handshake, se genera una clave de cifrado compartida que se usa para cifrar todo el tráfico.
- Se trata de engañar a la víctima para que reinstale una clave usada.
- El ataque es peor si se usa **WPA2-PSK (TKIP) o AES-GCMP**, ya que permite no sólo descifrar tráfico sino inyectar paquetes.
- El handshake de **WPA3** es resistente al ataque KRACK.

# Kali Linux y Aircrack



# Bloque vs. Flujo

## CIFRADO EN BLOQUE

El algoritmo de cifrado se aplica sobre cada bloque de mensaje usando la misma clave.

**Ventajas:**

* Alta difusión
* Imposible introducir bloques falsos sin detectarlo

**Desventajas:**

* Baja velocidad de cifrado
* Propenso a errores de cifrado: Un error se propaga a todo el bloque
* No oculta información estructurada

## CIFRADO EN FLUJO

El algoritmo de cifrado se aplica sobre cada bit de mensaje usando los distintos bits de la secuencia de clave.

**Ventajas:**

* Alta velocidad de cifrado
* No propaga errores
* Sí oculta información estructurada

**Desventajas:**

* Baja difusión
* Vulnerable a alteraciones

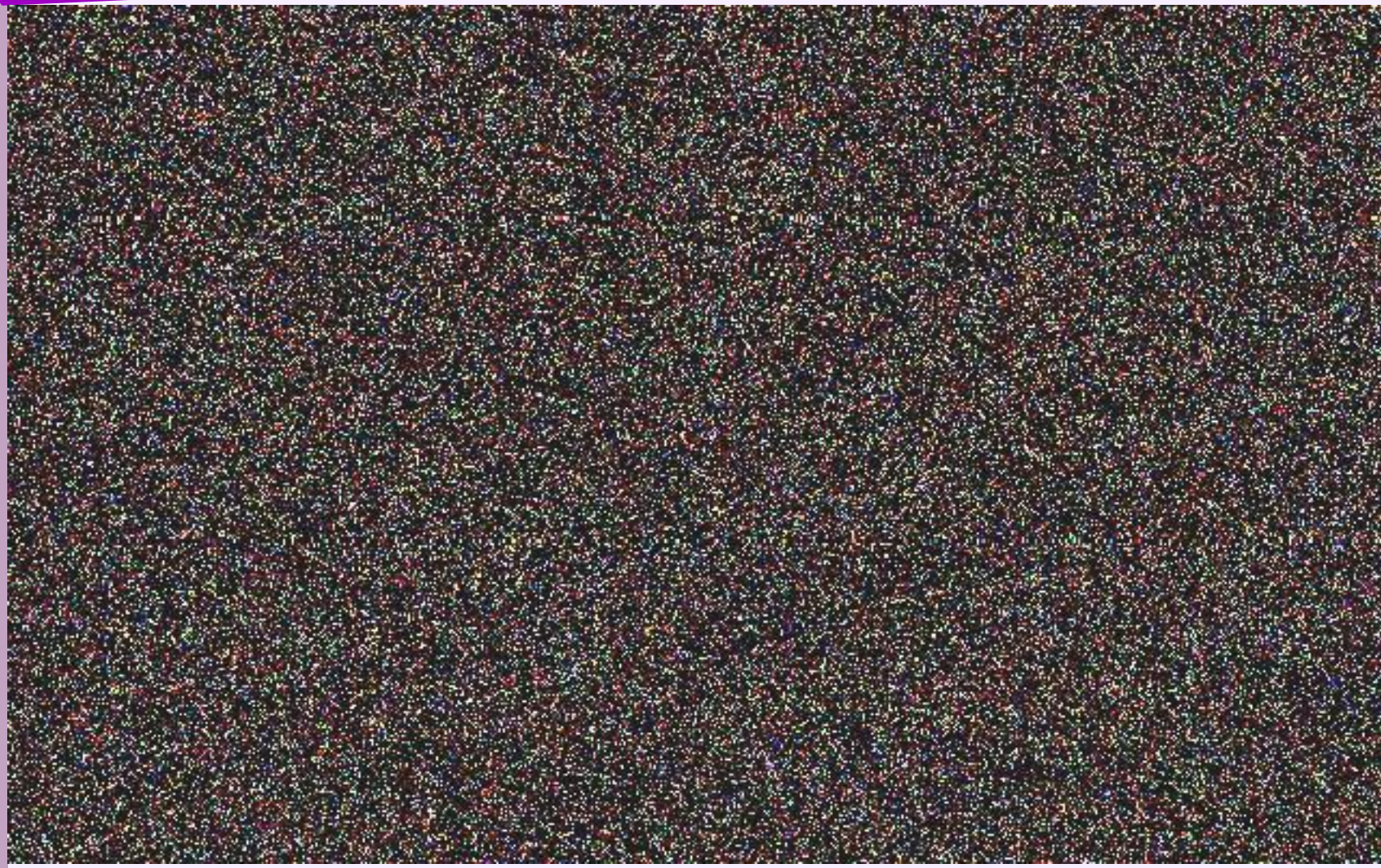
# Cifrado en Bloque vs. Flujo



# Cifrado en Bloque



# Cifrado en Flujo



# Modos de Cifrado en Bloque

•El problema es que las imágenes (y datos similares) contienen mucha información que se repite en bloques grandes, y las repeticiones de esos bloques se quedan sin cambiar en el cifrado.

•Por eso, no es recomendable usar un cifrado en bloque en modo:

1. **ECB: Electronic Code Book**

•Normalmente siempre se usa el cifrado en bloque en uno de los modos que lo tratan como cifrado en flujo:

2. **CFB: Cipher Feedback**
3. **OFB: Output Feedback**
4. **CBC: Cipher Block Chaining**
5. **PCBC: Propagating Cipher Block Chaining**
6. **IGE: Infinite Garble Extension**
7. **CTR: Counter**

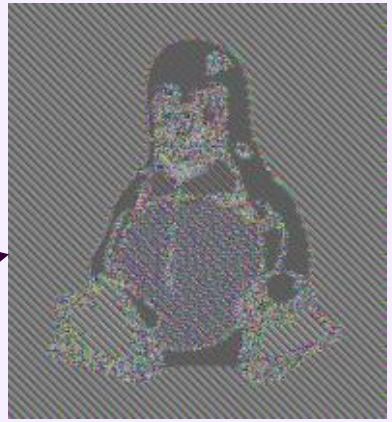
•Además hay **modos de cifrado en bloque autenticado**: GCM, GCMP, CCM y CCMP, CMAC… usados en Wi-Fi, TLS, BLE, WhatsApp…

# Modo ECB

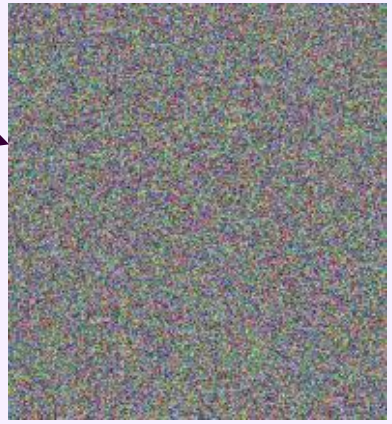
Electronic Code Book. Libro Electrónico de Códigos



**con ECB**

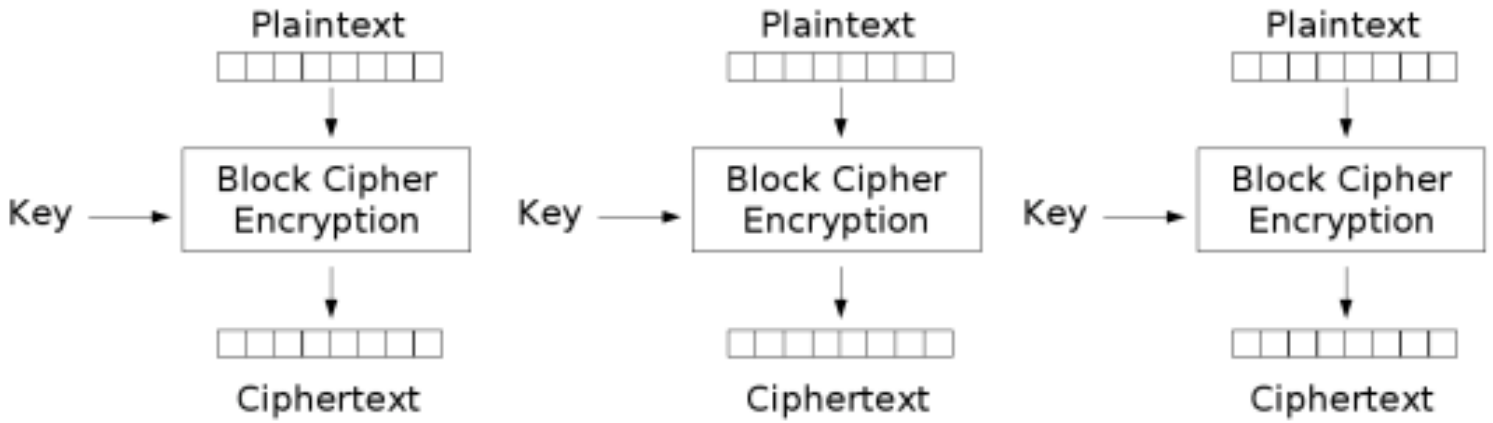


**con otros modos**



# Modo ECB

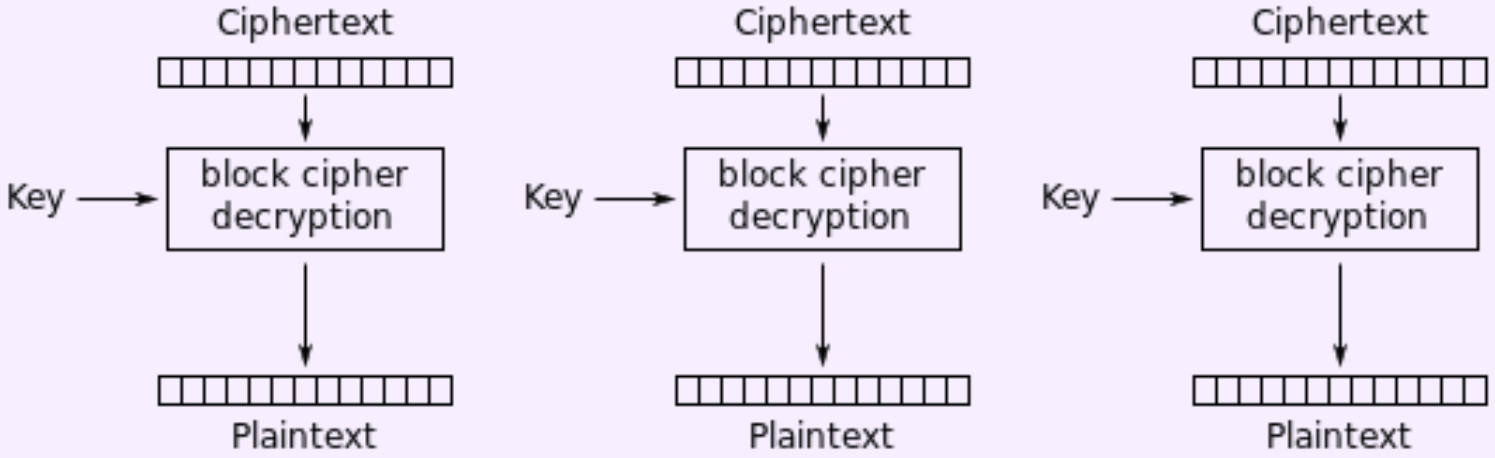
Electronic Code Book. Libro Electrónico de Códigos




Electronic Codebook (ECB) mode encryption

# Modo ECB

Electronic Code Book. Libro Electrónico de Códigos




Electronic Codebook (ECB) mode decryption

Ejemplo: Key=

| |
|---|
| 00→01 |
| 01→10 |
| 10→11 |
| 11→00 |

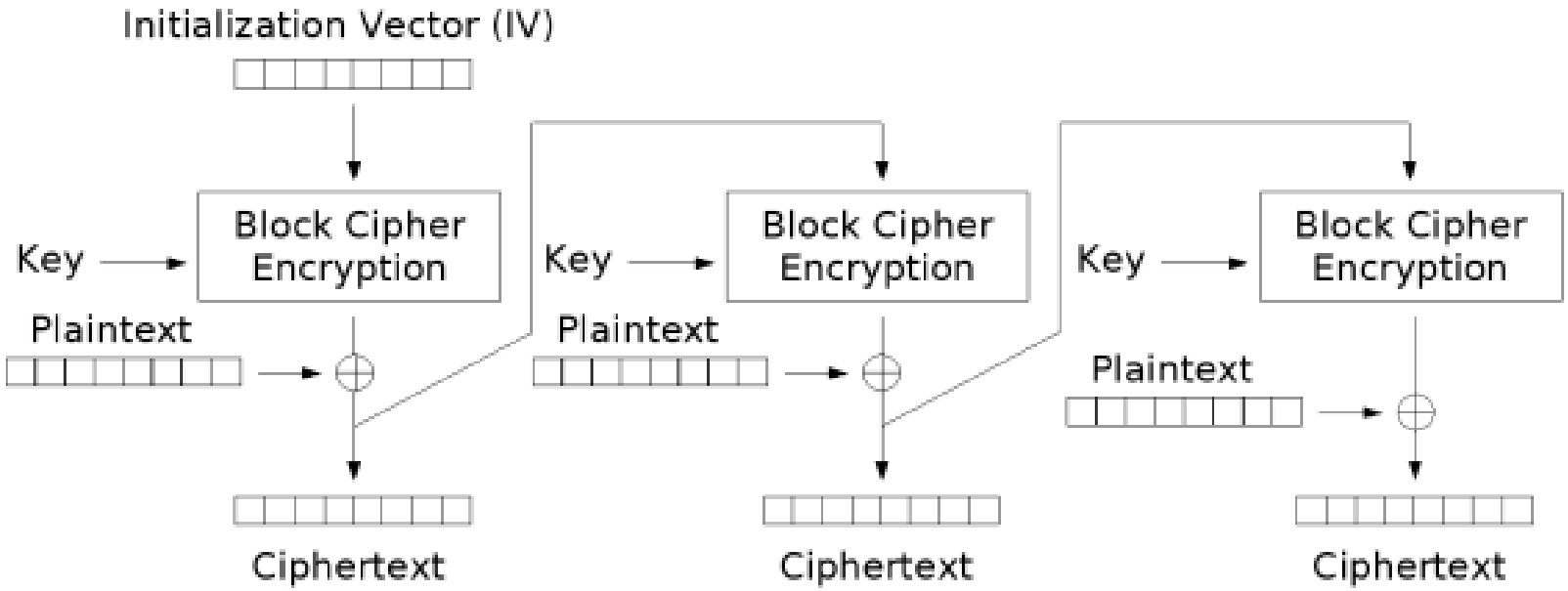
cipher table

# Modo ECB

- Los bloques originales idénticos producen bloques cifrados idénticos. Por tanto, siempre es posible reconocer patrones del texto original en el texto cifrado, lo que facilita ataques estadísticos. Por eso, no debe usarse sobre más de un bloque.
- Es vulnerable a ataques por modificación de texto cifrado porque ni siquiera serían detectados.
- Es vulnerable a ataques por repetición pues se puede sustituir, sin ser detectado, bloques cifrados con otros bloques cifrados generados con la misma clave.
- **Requiere rellenar** el último bloque.
- Se cifra cada bloque de texto original con la misma clave, de forma independiente. Por tanto, **cifrado y descifrado son paralelizables**.
- Los **errores de transmisión no se propagan** de unos bloques a otros.

# Modo CFB

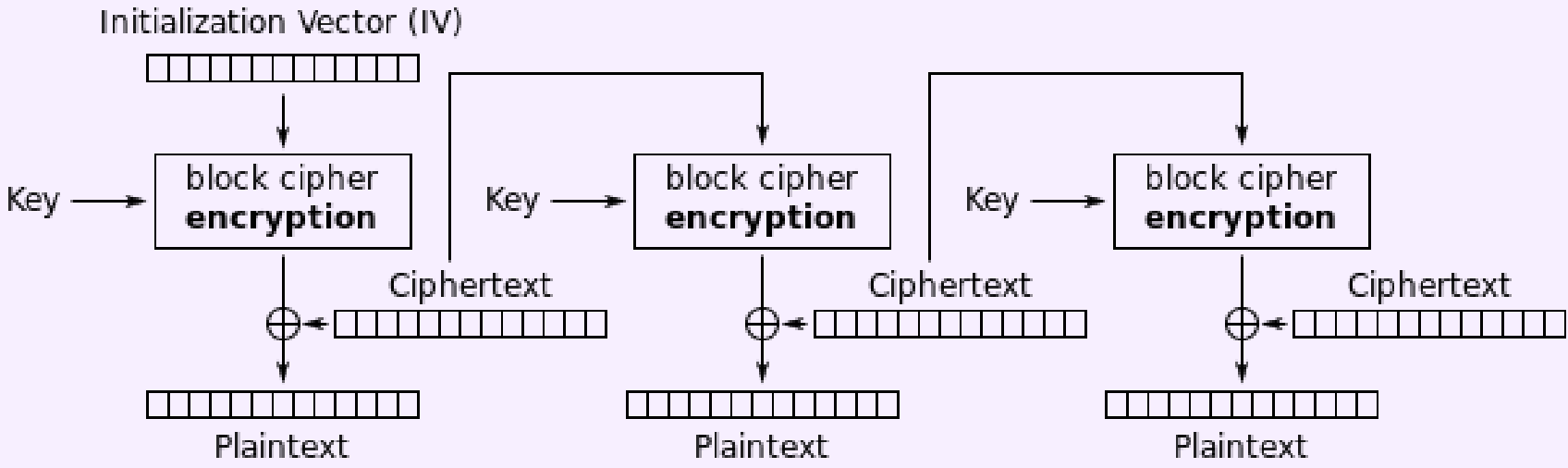
## Cipher FeedBack. Realimentación con Texto Cifrado




Cipher Feedback (CFB) mode encryption

# Modo CFB

## Cipher FeedBack. Realimentación con Texto Cifrado




Cipher Feedback (CFB) mode decryption

Ejemplo: IV=10 Key=

| |
|---|
| 00→01 |
| 01→10 |
| 10→11 |
| 11→00 |

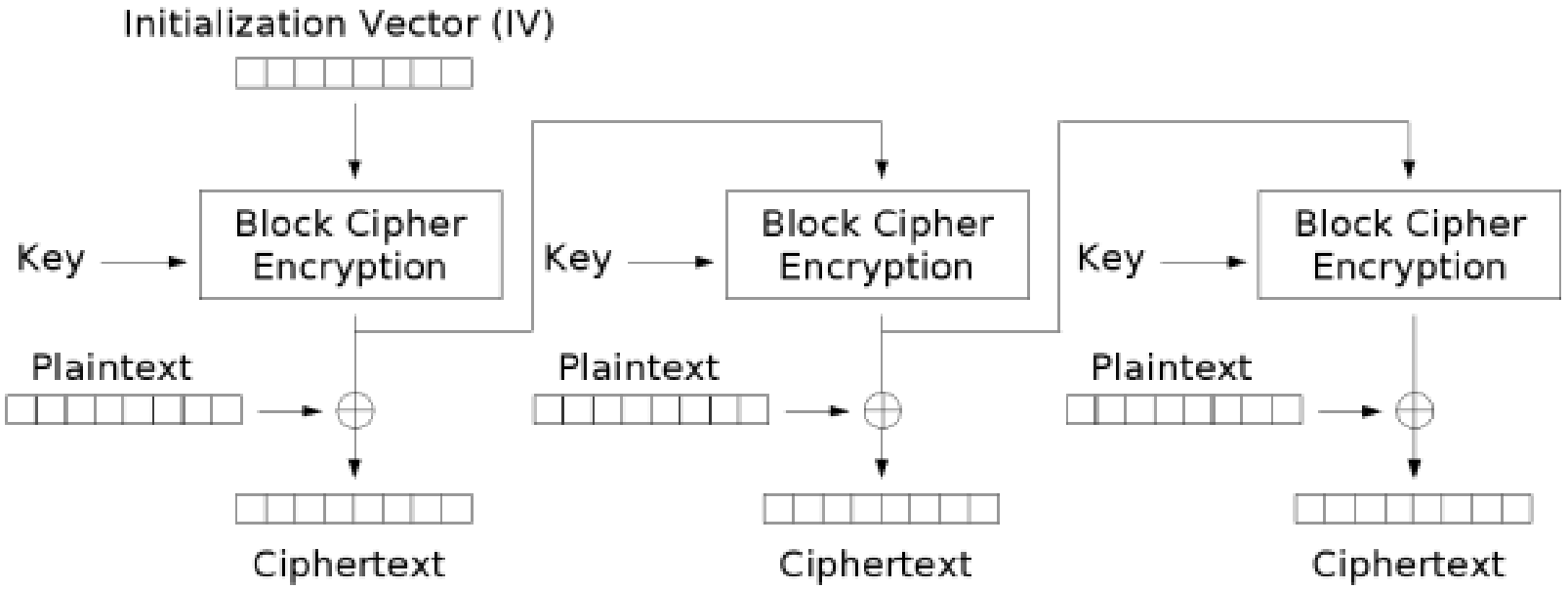
cipher table

# Modo CFB

- Los bloques de texto original no entran en el algoritmo de cifrado.
- **No requiere rellenar** el último bloque.
- Cada bloque te texto original se cifra haciendo una XOR con el bloque de secuencia cifrante obtenido con el algoritmo de cifrado alimentado con el bloque de texto cifrado anterior, luego el **cifrado no es paralelizable**.
- Para descifrar un bloque de texto cifrado hay que hacerle la XOR con el bloque de secuencia cifrante generado con el algoritmo de cifrado alimentado con el bloque anterior de texto cifrado, luego el **descifrado sí es paralelizable**.
- Los **errores de transmisión sí se propagan** de unos bloques a otros.
- Los bloques de secuencia cifrante pueden ser generados simultáneamente por emisor y receptor, de forma que cifrado y descifrado se pueden hacer prácticamente simultáneos, lo que lo hace ideal para aplicaciones en tº real.

# Modo OFB

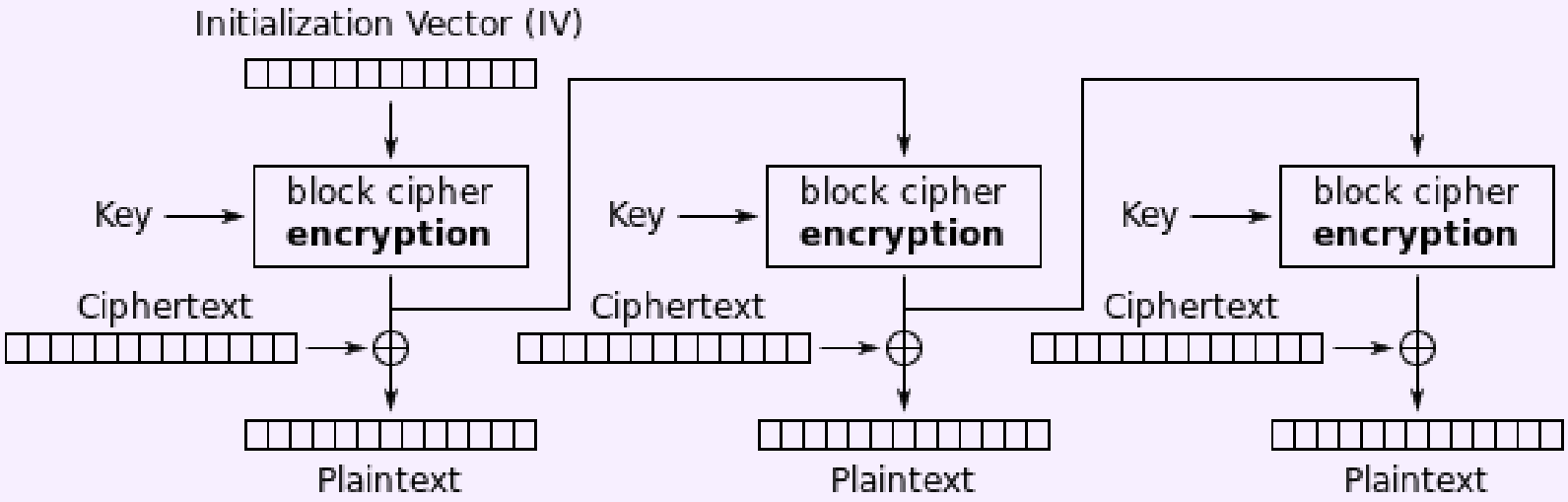
Output FeedBack. Realimentación con Salida




Output Feedback (OFB) mode encryption

# Modo OFB

## Output FeedBack. Realimentación con Salida



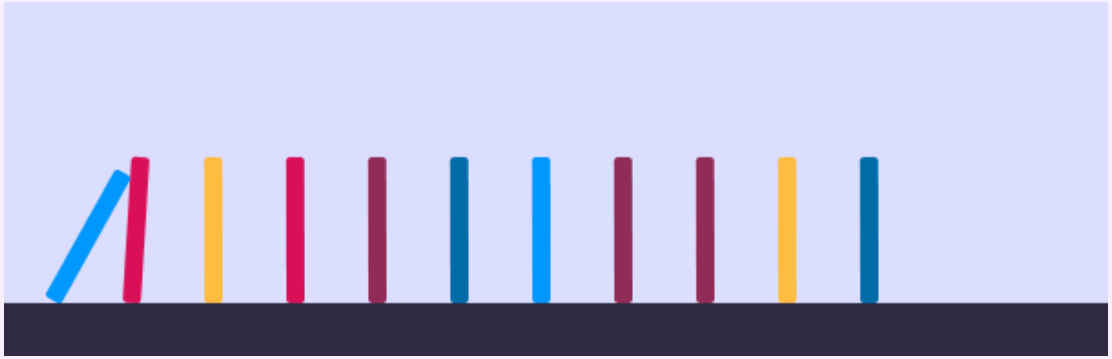

Output Feedback (OFB) mode decryption

Ejemplo: IV=10 Key=

| cipher table |
|---|
| 00→01 |
| 01→10 |
| 10→11 |
| 11→00 |

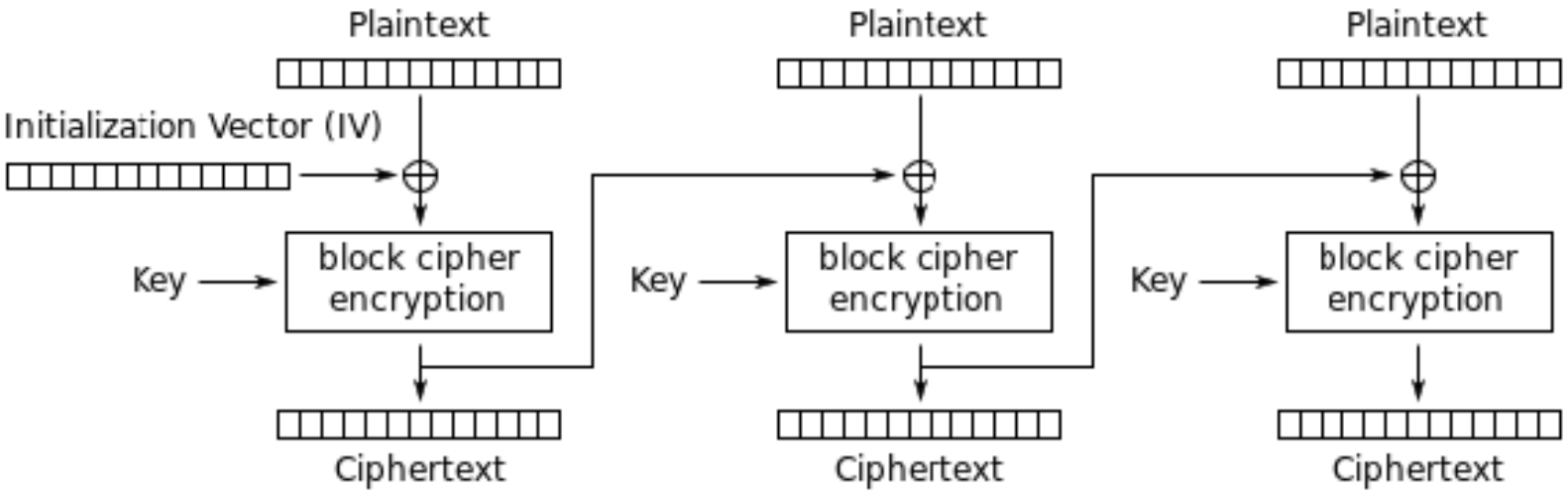
# Modo OFB

- Los bloques de texto original no entran en el algoritmo de cifrado.
- **No requiere relleno**.
- Todos los bloques obtenidos dependen de lo anterior, así que en general **cifrado y descifrado no son paralelizables**, pero como los bloques anteriores solo se usan en la XOR final, el algoritmo se puede ejecutar con antelación, lo que permite cifrar o descifrar en paralelo una vez se tengan los bloques originales o cifrados.
- Los **errores de transmisión no se propagan** de unos bloques a otros.



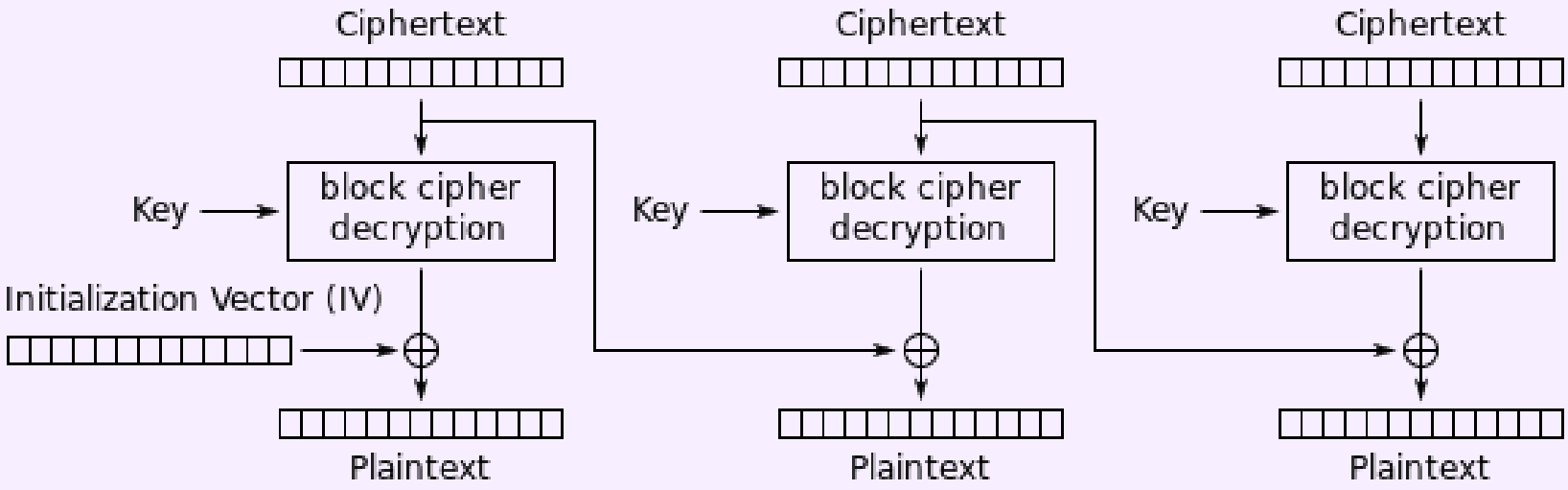
# Modo CBC

Cipher Block Chaining. Encadenamiento de Bloques Cifrados



# Modo CBC

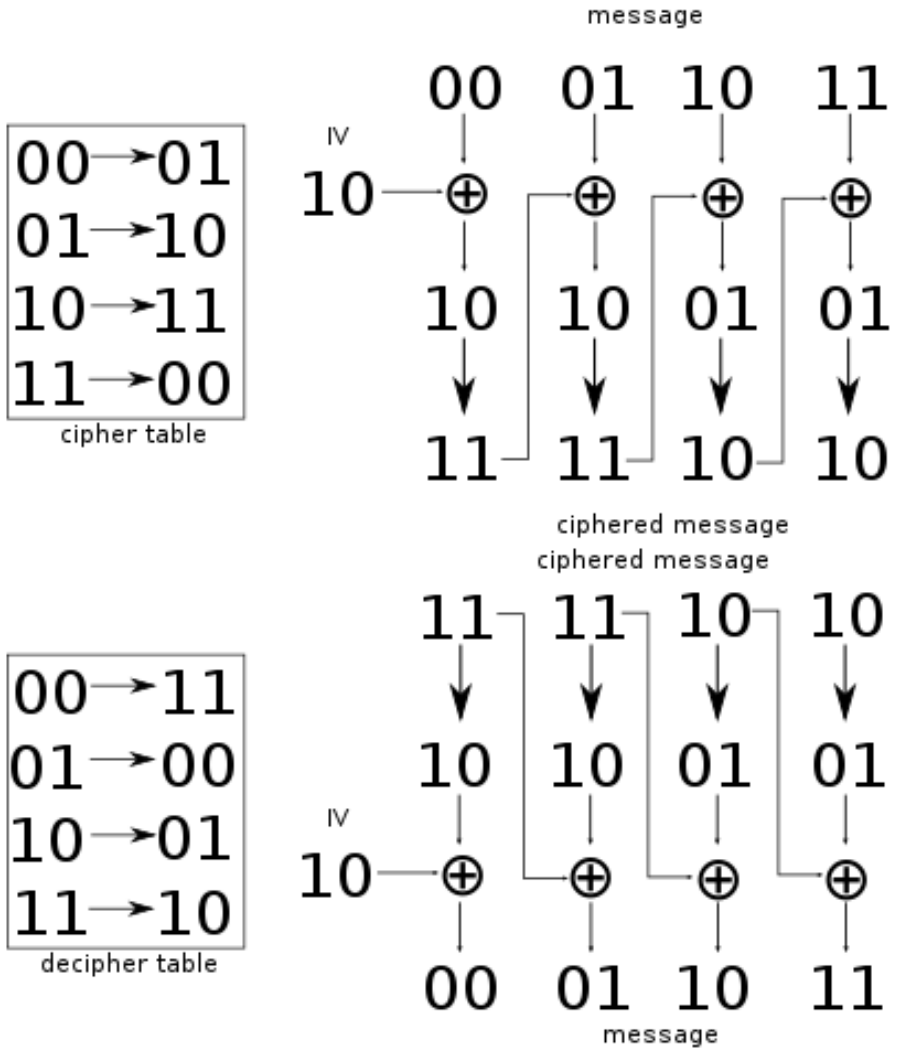
Cipher Block Chaining. Encadenamiento de Bloques Cifrados




Cipher Block Chaining (CBC) mode decryption

# Modo CBC

**Ejemplo**



# Modo CBC











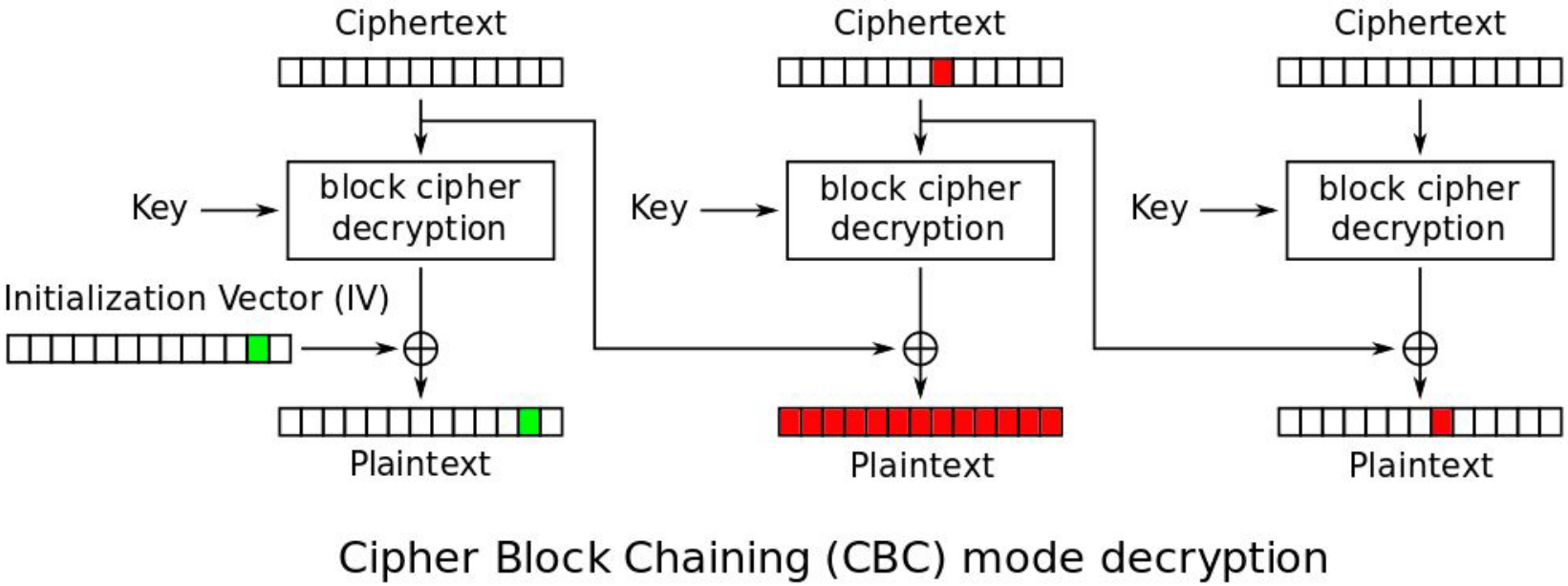



- La entrada al cifrado es el resultado de la operación XOR entre el actual bloque original y el bloque cifrado anterior, usando la misma clave.
- Para descifrar, primero hay que descifrar un bloque cifrado y luego hacer la XOR con el bloque cifrado anterior.
- **Es el modo más usado**, aunque en aplicaciones en tiempo real tiene problemas de eficiencia.
- Solo da seguridad IND-CPA (INDistinguishability under Chosen Plaintext Attack).
- **Requiere relleno**, aunque se puede evitar con la técnica cipher stealing.
- El **cifrado no es paralelizable**.
- El descifrado de un bloque cifrado no depende de los anteriores, sino sólo de su posición, luego el **descifrado sí es paralelizable**.
- Los **errores de transmisión sí se propagan** de unos bloques a otros.



# Ataques al Modo CBC

## Ataque bit flipping

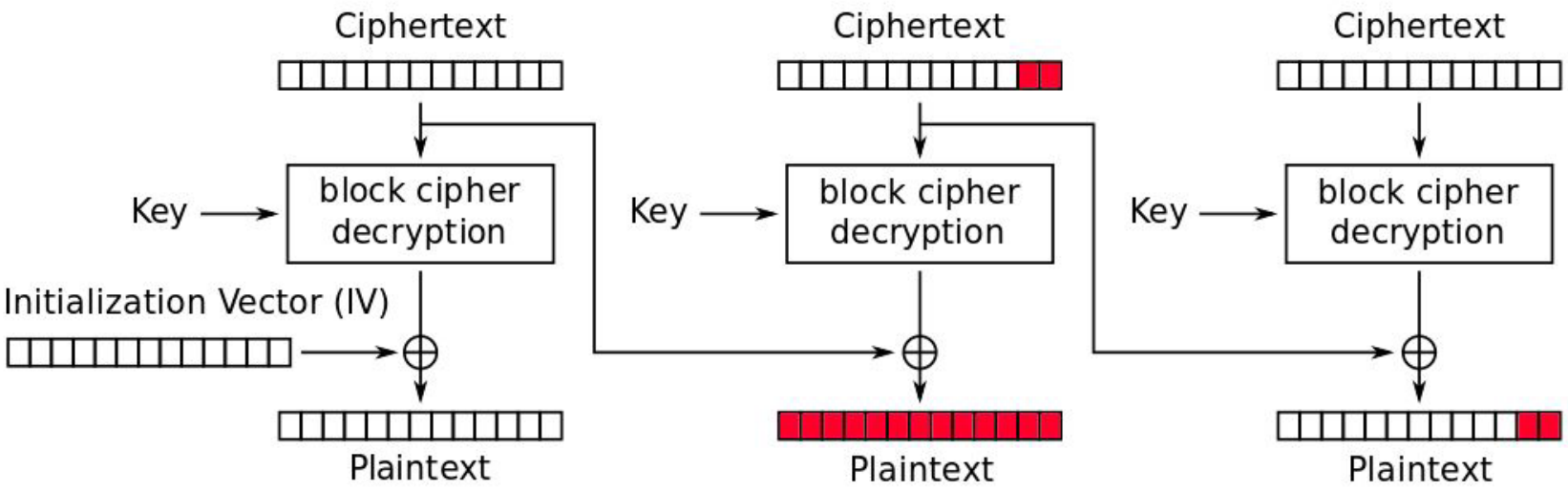



Cipher Block Chaining (CBC) mode decryption



# Ataques al Modo CBC

## Ataque padding oracle

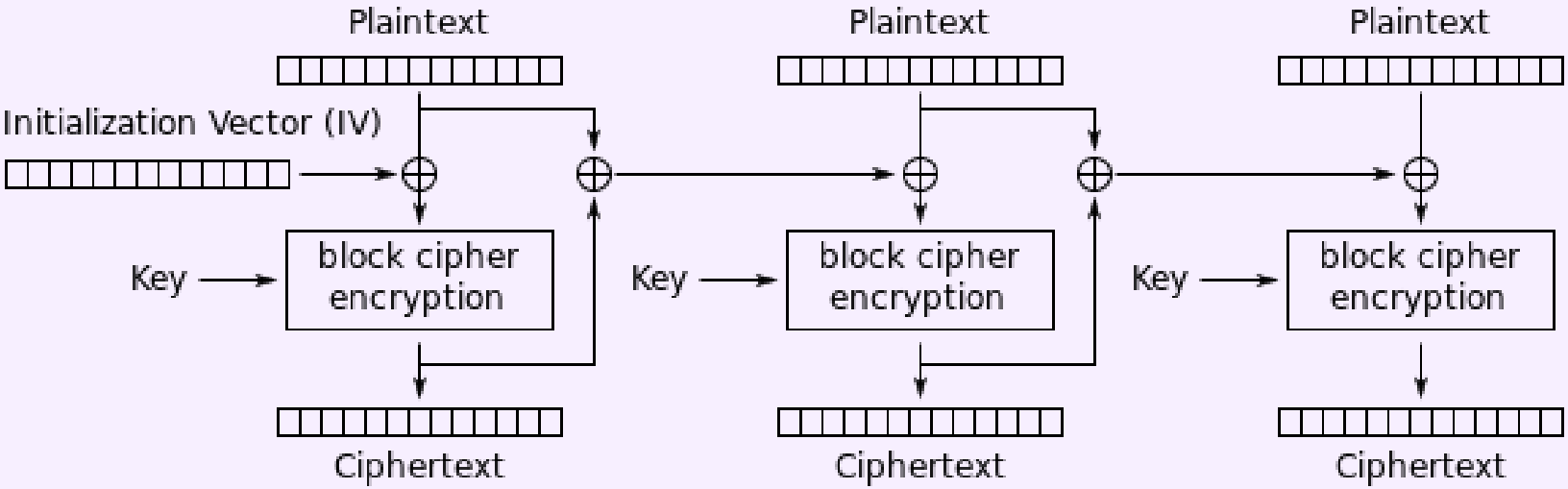



Cipher Block Chaining (CBC) mode decryption



# Modo PCBC

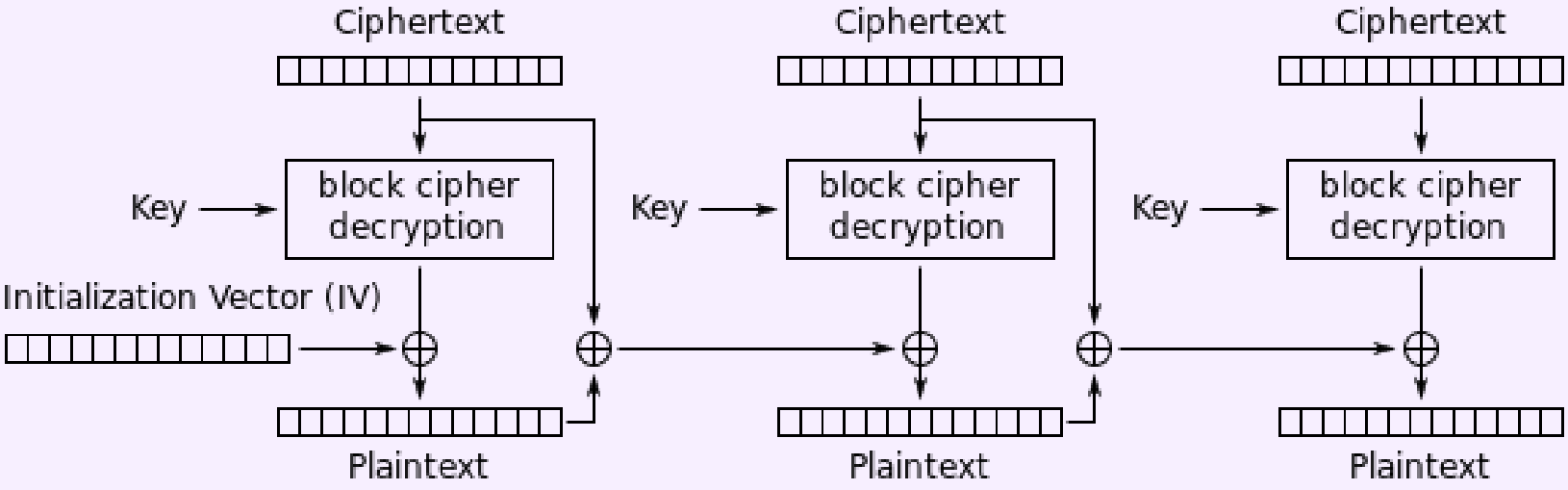
**Propagating Cipher Block Chaining.** Encadenamiento de Propagación de Bloques Cifrados




Propagating Cipher Block Chaining (PCBC) mode encryption

# Modo PCBC

**Propagating Cipher Block Chaining.** Encadenamiento de Propagación de Bloques Cifrados




Propagating Cipher Block Chaining (PCBC) mode decryption

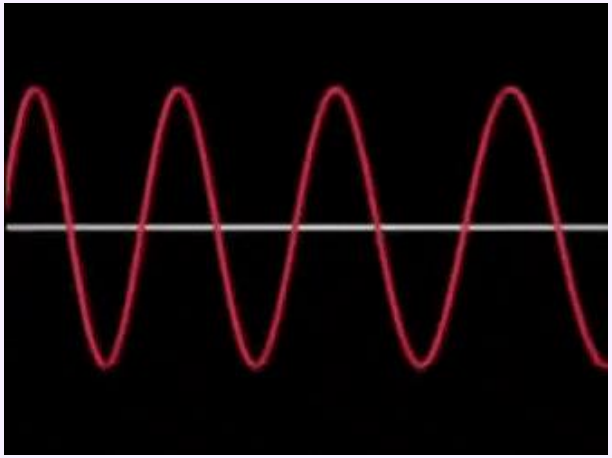
Ejemplo: IV=10 Key=

| | |
|---|---|
| 00 | 01 |
| 01 | 10 |
| 10 | 11 |
| 11 | 00 |

cipher table

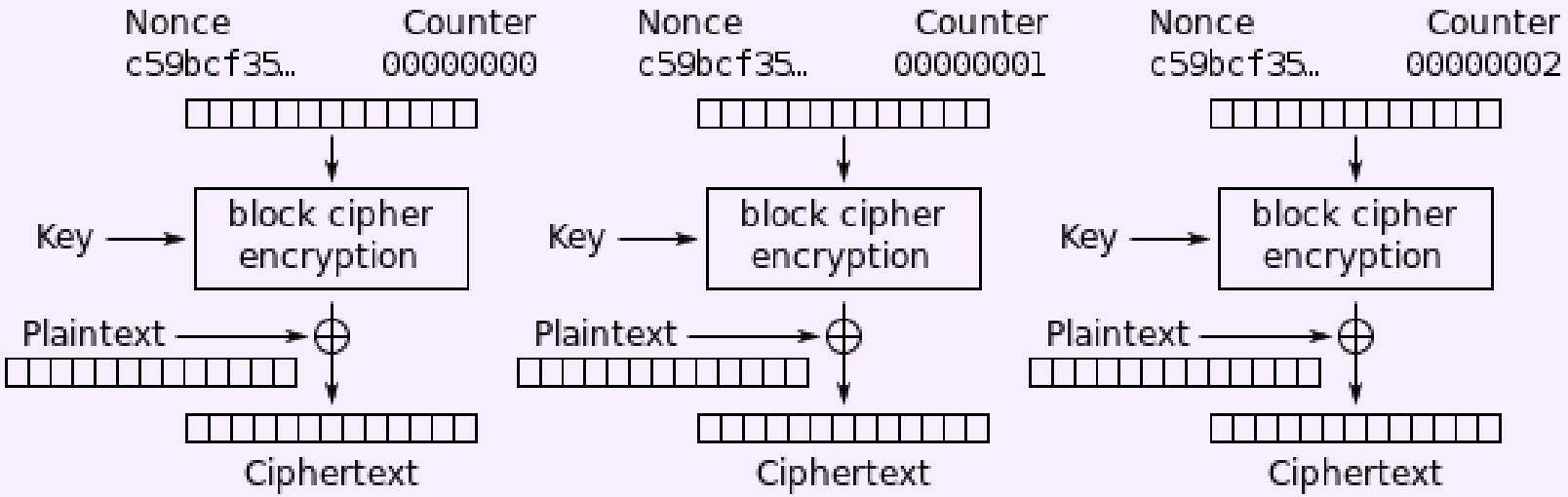
# Modo PCBC

- **Requiere relleno**.
- El **cifrado no es paralelizable**.
- El **descifrado tampoco es paralelizable**.
- Los **errores de transmisión se propagan** de unos bloques a otros.



# Modo CTR

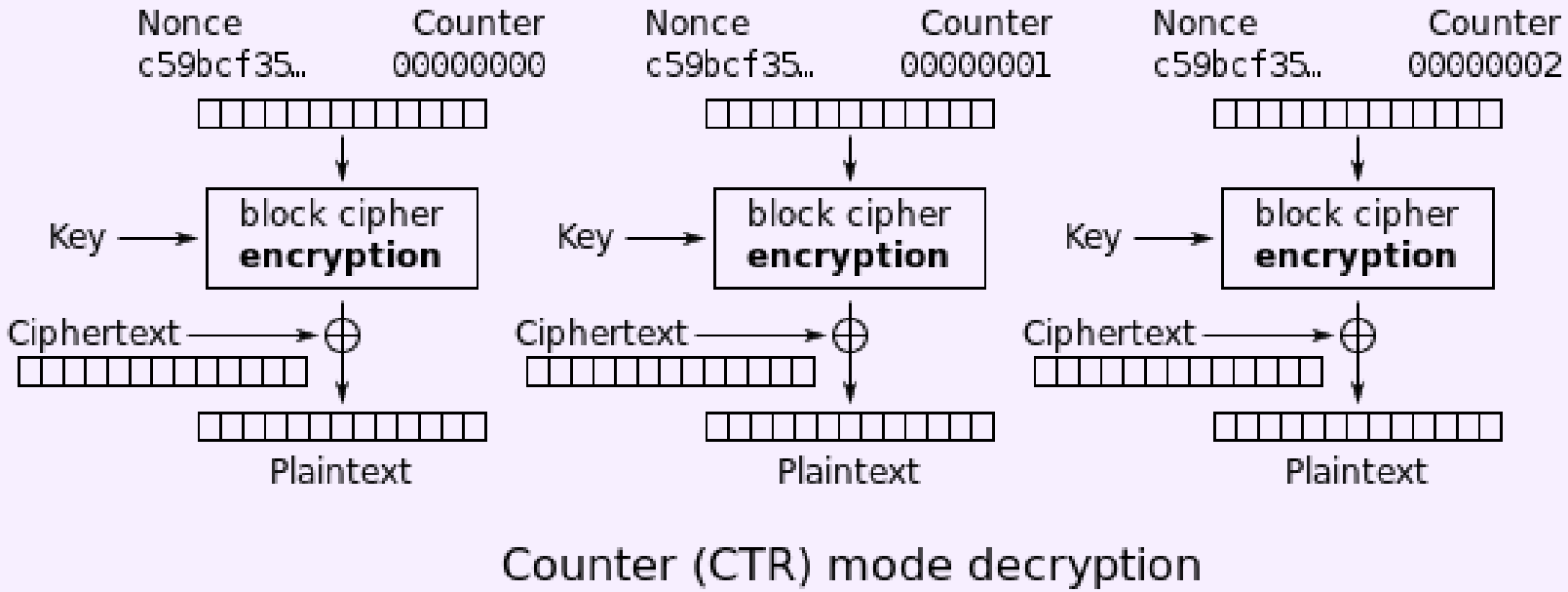
## Modo Contador.




Counter (CTR) mode encryption

# Modo CTR

## Modo Contador.




Counter (CTR) mode decryption

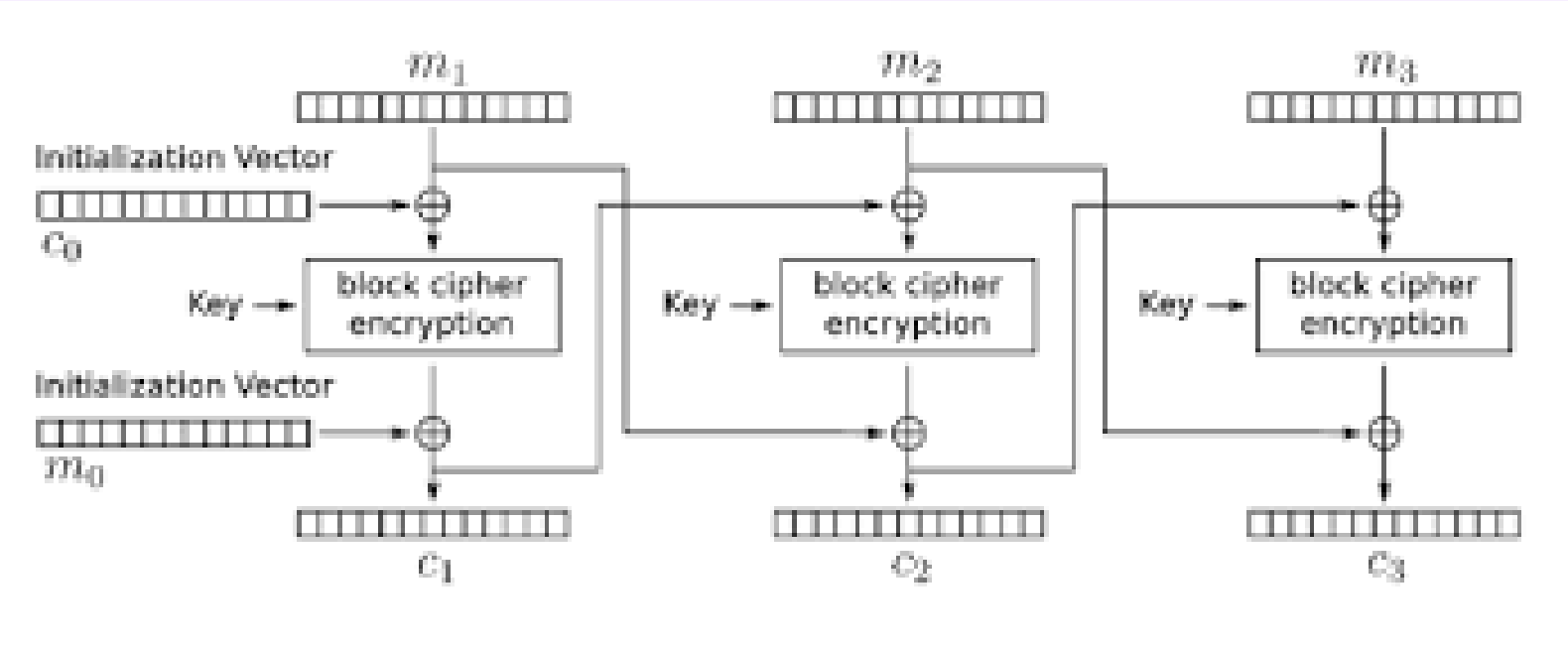
# Modo CTR

- Los bloques de texto original no entran en el algoritmo de cifrado.
- Los bloques de texto original idénticos no resultan en bloques de texto cifrado idénticos.
- **No requiere relleno**.
- **Cifrado y descifrado son paralelizables**.
- Los **errores de transmisión no se propagan** de unos bloques a otros.



# Modo IGE

**Infinite Garble Extension.** Extensión Infinita Garble

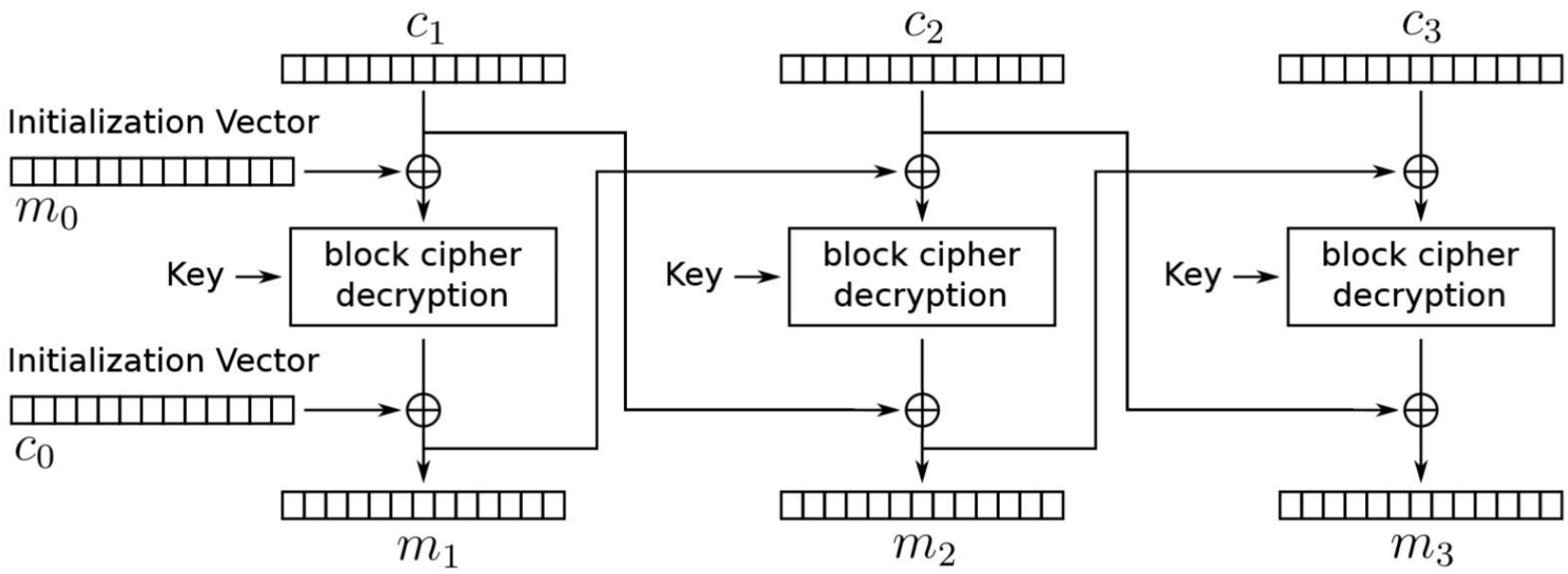



Telegram

CryptULL

# Modo IGE

**Infinite Garble Extension.** Extensión Infinita Garble




Telegram

# Modo IGE

- No está incluido en los modos recomendados por el NIST.
- Está roto para su uso en autenticación cuando se usa la misma clave en varios mensajes, pero no es el caso de Telegram.
- **Sí requiere relleno**.
- **Ni cifrado ni descifrado son paralelizables**.
- Los **errores de transmisión sí se propagan** de unos bloques a otros.



# Comparación de Modos de Cifrado en Bloque para Secreto

| | ECB | CFB | OFB | CBC | PCBC | CTR |
|---|---|---|---|---|---|---|
| Cifrado paralelizable | Sí | No | No | No | No | Sí |
| Descifrado paralelizable | Sí | Sí | No | Sí | No | Sí |
| Acceso de lectura aleatorio | Sí | Sí | No | Sí | No | Sí |

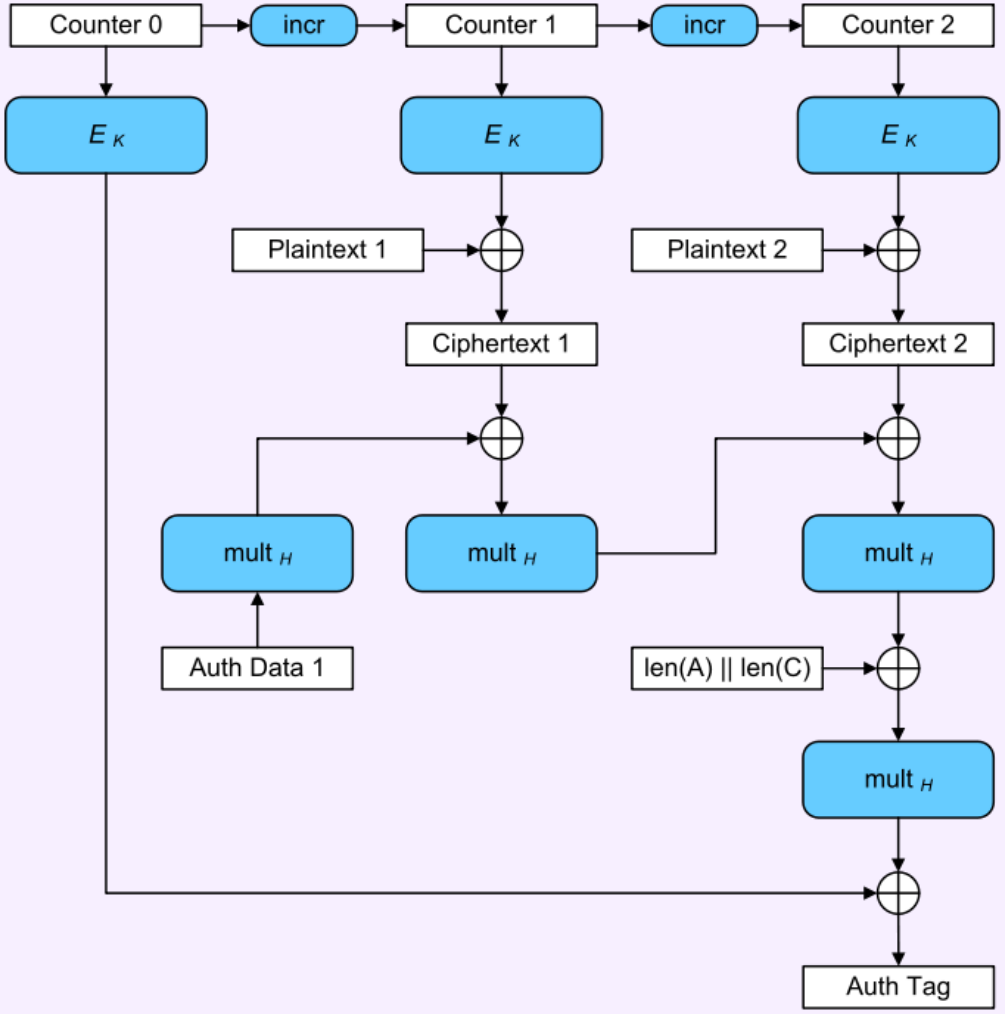
| Modo | Efecto de errores in el texto cifrado $C_i$ | Efecto de errores in el vector de inicialización IV |
|---|---|---|
| **ECB** | Errores aleatorios en $P_i$ | No aplicable |
| **CFB** | Errores específicos en $P_i$<br>Errores aleatorios en todos los bloques siguientes | Errores aleatorios en el descifrado de $C_1$, $C_2$, .., $C_j$ para algúnj entre 1 y b/s |
| **OFB** | Errores específicos en $P_i$ | Errores aleatorios en el descifrado de $C_1$, $C_2$, .., $C_n$ |
| **CBC** | Errores aleatorios en $P_i$<br>Errores específicos en $P_{i+1}$ | Errores específicos en el descifrado de $C_1$ |
| **CTR** | Errores específicos en $P_i$ | No aplicable* (*: errores en el bloque contador, $T_j$, resultando en errores aleatorios en el descifrado de $C_j$) |

# Modo Autenticado

## GCM:

## Galois Counter Mode.

- $mult_H$ es la multiplicación en $GF(2^{128})$ definido con el polinomio $x^{128}+x^7+x^2+x+1$, por la clave hash H obtenida aplicando el cifrado sobre un bloque de ceros.
- Auth Data 1 se obtiene aplicando el cifrado sobre un bloque de ceros.
- A es la longitud del texto autenticado y C es la del texto cifrado
- Además de proteger confidencialidad permite proteger integridad.
- No requiere relleno.
- No se propagan los errores entre bloques




https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf

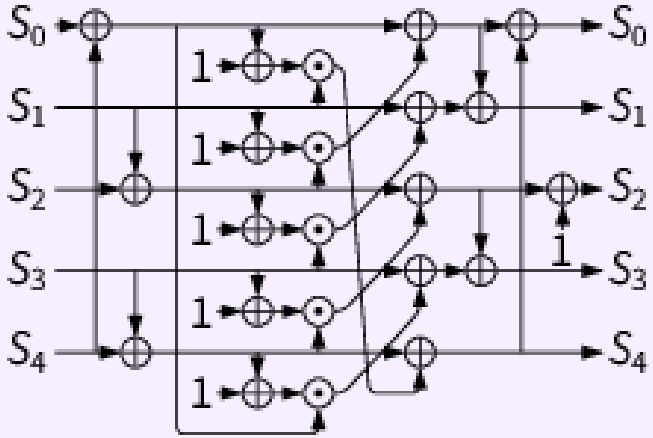
# Modos en las Tecnologías

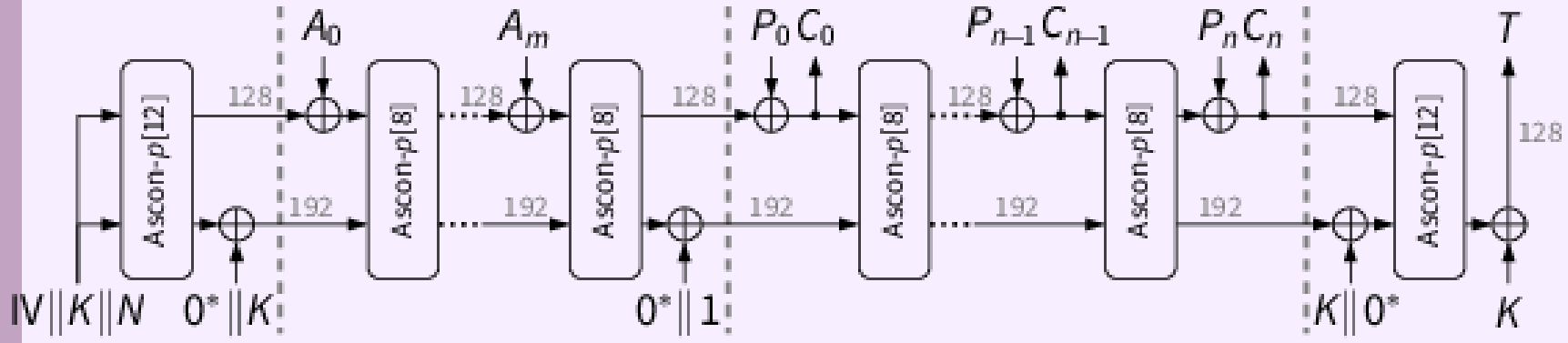
- **Telefonía móvil 3G: KASUMI-CBC**
- **DNI-e: AES-CBC**
- **SSL/TLS: AES-CCM**, que es CBC-MAC para confidencialidad y autenticación
- **BLE: AES-CCM**
- **TOR: AES-CCM**
- **Wi-Fi: AES-CCMP**, que combina **AES-CCM** con **AES-CTR**
- **Telefonía móvil 4G: AES-CTR**
- **Telefonía móvil 5G: AES-GCM**, que usa **AES-CTR** y autenticación de Galois
- **WhatsApp y Post-Cuántica: AES-256-GCM**
- **Kerberos: AES-PCBC**
- **Telegram: AES-IGE**

# ASCON –Nuevo Estándar de Criptografía Ligera

- Seleccionado por el NIST en 2023 y **estandarizado en 2025** (SP 800-232).
- Para proteger dispositivos **IoT** (sensores médicos, etiquetas RFID, smart cards) que no tienen potencia para ejecutar AES-GCM eficientemente.
- Basado en una construcción de **esponja**, similar a la función hash estándar SHA-3 (Keccak).
- Cifra y **autentica** en un solo proceso, garantizando confidencialidad e integridad.
- Utiliza un estado de 320 bits, dividido en 5 registros de 64 bits cada uno.
- Fases de la Esponja:
  - Absorción: Se introduce el Nonce y la Clave.
  - Permutación: Una serie de rondas de sustitución, difusión lineal y adición de constantes.
  - Exprimido: Se genera el texto cifrado y, al final, el Tag de Autenticación.

# ASCON


$S_0$
$S_1$
$S_2$
$S_3$
$S_4$
1
1
1
1
1
1
$S_0$
$S_1$
$S_2$
$S_3$
$S_4$


$A_0$
$A_m$
$P_0 C_0$
$P_{n-1} C_{n-1}$
$P_n C_n$
$T$
Ascon-$p$[12]
Ascon-$p$[8]
Ascon-$p$[8]
Ascon-$p$[8]
Ascon-$p$[8]
Ascon-$p$[12]
128
192
$IV \| K \| N$
$0^* \| K$
$0^* \| 1$
$K \| 0^*$
$K$
Initialization
Associated Data
Plaintext
Finalization